

ROYAUME DU MAROC

Office de la Formation Professionnelle et de la Promotion du Travail

Logo
OFPPT

Logo
Centre
Al Kendi

Logo
Filière
DAI

PROJET DE FIN D'ÉTUDES



PulmoScan AI

*Application mobile de pré-diagnostic des pathologies pulmonaires
par intelligence artificielle embarquée*

Centre de Formation Al Kendi — Casablanca

Brevet de Technicien Supérieur (BTS)

Filière : Développement des Applications Informatiques

Réalisé par :

Foudali Kenza
El Hajji Adnane

Encadré par :

Mme. Amina Aboulmira

Année universitaire 2025/2026

Dédicace

*À nos chers parents,
qui n'ont jamais cessé de croire en nous,
et dont les sacrifices silencieux ont éclairé chaque étape de notre parcours.*

*À nos frères et sœurs,
sources inépuisables de soutien et d'encouragement.*

*À nos professeurs,
qui ont semé en nous le goût du savoir et la rigueur du travail.*

*À tous nos amis,
avec qui nous avons partagé les joies et les difficultés de ces années de formation.*

*À tous ceux qui, de près ou de loin,
ont contribué à faire de ce projet une réalité,
nous dédions ce modeste travail.*

Kenza & Adnane

Remerciements

Avant toute chose, nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Nous adressons nos remerciements les plus sincères à notre encadrante, **Mme. Amina Aboulmira**, pour la qualité de son encadrement, sa disponibilité constante, ses conseils avisés et la confiance qu'elle nous a accordée tout au long de ce travail. Sa rigueur scientifique et son sens pédagogique ont été déterminants dans l'aboutissement de ce projet.

Nous remercions également l'ensemble du corps professoral et de l'équipe administrative du **Centre de Formation Al Kendi de Casablanca**, qui nous ont accompagnés durant ces deux années de formation au sein de la filière Développement des Applications Informatiques. La qualité de l'enseignement reçu a constitué le socle indispensable à la réussite de ce projet.

Nos remerciements vont aussi aux membres du jury, qui nous font l'honneur d'évaluer ce travail. Nous espérons que ce rapport saura répondre à leurs attentes et susciter leur intérêt.

Enfin, nous exprimons notre reconnaissance à nos familles et à nos amis, dont le soutien moral, la patience et les encouragements n'ont jamais fait défaut, particulièrement dans les moments les plus exigeants de ce parcours.

À toutes et à tous, nous disons : *merci*.

Résumé

PulmoScan AI est une application mobile multiplateforme, développée avec le framework Flutter, destinée à assister le personnel médical dans le pré-diagnostic des pathologies pulmonaires à partir de radiographies thoraciques. L'application embarque deux modèles d'intelligence artificielle exécutés localement grâce à TensorFlow Lite : un réseau de classification **DenseNet121** (issu de la bibliothèque TorchXRayVision et entraîné sur le jeu de données NIH ChestX-ray14, comprenant 112 120 radiographies annotées) capable de détecter 14 pathologies pulmonaires, et un réseau de segmentation **U-Net** qui isole les champs pulmonaires sur l'image.

La particularité fondamentale de PulmoScan AI réside dans son architecture *offline-first* : l'intégralité du traitement — acquisition de l'image, inférence des modèles, stockage des données et génération des rapports PDF — s'effectue sur l'appareil, sans aucune dépendance à un serveur distant. Les données des patients sont conservées dans une base de données locale SQLite, et l'authentification repose sur un hachage SHA-256 des mots de passe. Cette approche garantit la confidentialité des données médicales, leur disponibilité hors connexion et une conformité accrue en matière de protection des données personnelles.

Ce rapport présente le contexte du projet, la conception détaillée du système selon une démarche Scrum adaptée, l'implémentation technique du pipeline d'inférence, les interfaces réalisées et les campagnes de tests menées pour valider la fiabilité de la solution.

Mots-clés : Intelligence artificielle, radiographie thoracique, DenseNet121, U-Net, TensorFlow Lite, Flutter, SQLite, pré-diagnostic, offline-first, santé numérique.

Abstract

PulmoScan AI is a cross-platform mobile application, built with the Flutter framework, designed to assist medical staff in the pre-diagnosis of pulmonary pathologies from chest X-rays. The application embeds two artificial intelligence models running locally through TensorFlow Lite : a **DenseNet121** classification network (from the TorchXRayVision library, trained on the NIH ChestX-ray14 dataset of 112,120 annotated radiographs) capable of detecting 14 lung pathologies, and a **U-Net** segmentation network that isolates the lung fields in the image.

The core distinguishing feature of PulmoScan AI is its *offline-first* architecture : the entire workflow — image acquisition, model inference, data storage and PDF report generation — runs on-device, with no dependency on any remote server. Patient data is stored in a local SQLite database, and authentication relies on SHA-256 password hashing. This approach guarantees the confidentiality of medical data, its offline availability and improved compliance with personal data protection requirements.

This report presents the project context, the detailed system design following an adapted Scrum methodology, the technical implementation of the inference pipeline, the developed interfaces and the testing campaigns carried out to validate the reliability of the solution.

Keywords : Artificial intelligence, chest X-ray, DenseNet121, U-Net, TensorFlow Lite, Flutter, SQLite, pre-diagnosis, offline-first, digital health.

Résumé en arabe

(Cette page contient le résumé en langue arabe.)

Note technique

Le résumé en arabe nécessite une compilation avec **XeLaTeX** et un moteur de support bidirectionnel pour un affichage correct des caractères arabes. Afin de garantir la compilation propre du présent document avec **pdfLaTeX**, le texte arabe n'a pas été intégré ici. Le contenu correspond à la traduction du résumé français : présentation de l'application PulmoScan AI, de son architecture hors-ligne, des deux modèles d'intelligence artificielle (DenseNet121 et U-Net) et de l'approche de confidentialité des données médicales.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Résumé en arabe	v
Liste des figures	xi
Liste des tableaux	xii
Liste des abréviations	xiii
Introduction Générale	1
1 Le contexte général du projet	3
Introduction	3
1.1 Contexte et Objectifs	3
1.2 Problématique	4
1.3 Solutions Proposées	4
1.4 Notre Solution Développée	5
1.5 Fonctionnalités de l'Application	5
1.5.1 Écran d'Accueil et Connexion	6
1.5.2 Onboarding	6
1.5.3 Gestion des Patients	6
1.5.4 Création d'Examen	6
1.5.5 Analyse par Intelligence Artificielle (DenseNet121)	6
1.5.6 Segmentation Pulmonaire (U-Net)	7
1.5.7 Visualisation des Résultats	7
1.5.8 Tableau de Bord	7
1.5.9 Exigences Non Fonctionnelles	7
1.6 Étude de l'Existant	8
1.6.1 CheXpert (Stanford University)	8

1.6.2	Infermedica / Ada Health	8
1.6.3	Valeur Ajoutée de PulmoScan AI	8
	Conclusion	9
2	Conception et Modélisation	10
	Introduction	10
2.1	Méthodologies Adoptées	10
2.1.1	Approche Scrum Adaptée	10
2.1.2	Architecture Offline-First	10
2.2	Gestion de Projet	11
2.2.1	Sprints Scrum	11
2.2.2	Diagramme de Gantt	11
2.2.3	Diagramme PERT	12
2.3	Conception	13
2.3.1	Identification des Acteurs	13
2.3.2	Acteurs du Système PulmoScan AI	13
2.3.3	Diagramme de Cas d'Utilisation	13
2.3.4	Diagramme de Classes	14
2.3.5	Diagrammes de Séquence	14
2.3.6	Diagramme d'Activité	17
2.3.7	Diagramme d'État-Transition : Objet Examen	18
2.3.8	Modèle Conceptuel de Données (MCD)	18
2.3.9	Modèle Logique de Données (MLD)	19
	Conclusion	19
3	Implémentation et Déploiement	20
	Introduction	20
3.1	Outils et Technologies Utilisés	20
3.1.1	Frontend Mobile — Flutter & Dart	22
3.1.2	Intelligence Artificielle — TensorFlow Lite	23
3.1.3	Modèles IA — TorchXRayVision	24
3.1.4	Base de Données Locale — SQLite	25
3.1.5	Traitement d'Image — package <code>image</code>	26
3.1.6	Acquisition d'Image — <code>image_picker</code>	26
3.1.7	Génération de Rapports PDF	27
3.1.8	Sécurité — SHA-256 via <code>crypto</code>	28
3.1.9	Gestion de Session — <code>SharedPreferences</code>	28
3.1.10	Backend Optionnel — Node.js & Express.js	29
3.1.11	Environnement de Développement — Visual Studio Code	30
3.1.12	Contrôle de Version — Git & GitHub	31

3.1.13	Entraînement et Conversion des Modèles — Google Colab . . .	31
3.1.14	Authentification — SHA-256 & SharedPreferences	33
3.1.15	Backend Optionnel — Node.js / Express / Railway	33
3.1.16	Outils de Développement & DevOps	34
3.2	Structure du Projet Flutter	34
3.3	Pipeline d'Inférence IA Complet	36
3.3.1	Prétraitement des Images	36
3.3.2	Inférence DenseNet121	36
3.3.3	Post-traitement et Sélection du Diagnostic	37
3.3.4	Segmentation U-Net	38
3.4	Sécurité et Confidentialité	38
	Conclusion	38
4	Présentation des Interfaces	40
	Introduction	40
4.1	Écrans d'Onboarding	40
4.2	Landing Page	42
4.3	Connexion & Inscription	43
4.4	Dashboard & Statistiques	45
4.5	Liste des Patients	47
4.6	Détails Patient & Historique des Examens	49
4.7	Création d'Examen	50
4.8	Résultats de l'Analyse IA	52
4.9	Génération du Rapport PDF	54
4.10	Profil Médecin	54
	Conclusion	55
5	Tests et Assurance Qualité	56
	Introduction	56
5.1	Tests du Modèle IA	56
5.1.1	Validation de la Normalisation (Python)	56
5.1.2	Métriques de Performance par Pathologie	56
5.1.3	Comportement sur Images Synthétiques	57
5.2	Tests Fonctionnels	57
5.3	Tests d'Inférence sur Images Réelles NIH	58
5.3.1	Jeu de Données de Test	58
5.3.2	Résultats d'Inférence sur l'Application	59
5.4	Tests de l'API Backend (Postman)	61
5.5	Tests d'Interface et Ergonomie	63
5.6	Difficultés Rencontrées et Solutions	63

Conclusion	63
Conclusion Générale	65
Webographie / Références	66

Table des figures

1.1	Logo de l'application PulmoScan AI (widget ApertureMark)	5
2.1	Diagramme de Gantt	12
2.2	Diagramme PERT	12
2.3	Diagramme de cas d'utilisation	13
2.4	Diagramme de classes	14
2.5	Diagramme de séquence : Authentification	15
2.6	Diagramme de séquence : Création Patient	15
2.7	Diagramme de séquence : Acquisition Image	16
2.8	Diagramme de séquence : Analyse IA	17
2.9	Diagramme d'activité	17
2.10	Diagramme d'état-transition : Objet Examen	18
2.11	Modèle Conceptuel de Données (MCD)	18
4.1	Onboarding (1/3) — Capture	41
4.2	Onboarding (2/3) — Analyse IA	41
4.3	Onboarding (3/3) — Rapport	42
4.4	Landing page — Section hero du Centre Al Kendi	43
4.5	Écran de connexion avec formulaire e-mail/mot de passe	44
4.6	Inscription et vérification	45
4.7	Dashboard — Cartes statistiques : 3 patients, 17 examens, 17 résultats	46
4.8	Dashboard — Top diagnostics IA et répartition par statut	47
4.9	Liste des patients — Ahmed Benali, Fatima Zahra, Youssef Alaoui . . .	48
4.10	Fiche patient Ahmed Benali — Onglet Informations	49
4.11	Fiche patient Ahmed Benali — Onglet Historique (3 examens)	50
4.12	Création d'examen — Sélection du patient	51
4.13	Création d'examen — Image radiographique chargée, prête pour l'analyse IA	52
4.14	Résultats IA — Segmentation U-Net et diagnostic principal	53
4.15	Résultats IA — Histogramme des 14 pathologies (PROBABILITÉS) . .	53
4.16	Rapport PDF généré	54
4.17	Profil médecin — Dr. Kenza Foudali, réglages et déconnexion	55
5.1	Jeu de données NIH ChestX-ray14 — 14 classes sur Google Drive . . .	58

5.2	Résultat IA : Mass 97%, URGENT	60
5.3	Résultat IA : Atelectasis 77%, URGENT	60
5.4	Résultat IA : Cardiomegaly 97%, URGENT	61
5.5	Résultat IA : Emphysema 97%, URGENT	61
5.6	Test Postman — Authentification (POST /api/auth/login)	62
5.7	Test Postman — Récupération des patients (GET /api/patients)	62
5.8	Test Postman — Création d'examen (POST /api/examens)	62

Liste des tableaux

1.1	Exigences non fonctionnelles de PulmoScan AI	7
1.2	Étude comparative de l'existant	9
2.1	Backlog des sprints Scrum	11
3.1	Pile technologique de PulmoScan AI	21
3.2	Inventaire des écrans Flutter	35
3.3	Inventaire des services Flutter	35
3.4	Ordre exact des étiquettes TorchXRayVision (indices 0 à 13)	37
4.1	Patients de démonstration pré-chargés	48
5.1	Validation de la normalisation (tests Python)	56
5.2	AUC par pathologie (* = exclue du diagnostic principal)	57
5.3	Synthèse des tests fonctionnels	58
5.4	Résultats d'inférence sur radiographies réelles NIH	59
5.5	Difficultés rencontrées et solutions apportées	63

Liste des abréviations

Abréviation	Signification
AUC	Area Under the Curve (aire sous la courbe ROC)
API	Application Programming Interface
BPCO	Broncho-Pneumopathie Chronique Obstructive
BTS	Brevet de Technicien Supérieur
CIN	Carte d'Identité Nationale
CNN	Convolutional Neural Network (réseau de neurones convolutif)
CRUD	Create, Read, Update, Delete
DAI	Développement des Applications Informatiques
IA	Intelligence Artificielle
JWT	JSON Web Token
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
NIH	National Institutes of Health
PA	Paquets-Année (tabagisme)
PERT	Program Evaluation and Review Technique
PFE	Projet de Fin d'Études
RGPD	Règlement Général sur la Protection des Données
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SDK	Software Development Kit
SGBD	Système de Gestion de Base de Données
SHA	Secure Hash Algorithm
SQL	Structured Query Language
TFLite	TensorFlow Lite
TXV	TorchXRayVision
UML	Unified Modeling Language
UI/UX	User Interface / User Experience

Introduction Générale

Les maladies respiratoires figurent parmi les principales causes de morbidité et de mortalité à l'échelle mondiale. Selon l'Organisation mondiale de la santé, des affections telles que la pneumonie, la broncho-pneumopathie chronique obstructive (BPCO), l'emphysème ou les épanchements pleuraux touchent chaque année des centaines de millions de personnes. Au Maroc, les pathologies pulmonaires représentent une charge importante pour le système de santé, particulièrement dans les zones rurales et péri-urbaines où l'accès à un radiologue qualifié demeure limité.

La radiographie thoracique reste l'examen d'imagerie de première intention pour le dépistage de nombreuses pathologies pulmonaires : elle est rapide, peu coûteuse et largement disponible. Néanmoins, son interprétation requiert une expertise spécialisée qui n'est pas toujours présente sur le lieu de soin. Le délai entre l'acquisition de l'image et son interprétation par un spécialiste peut retarder la prise en charge du patient, avec des conséquences cliniques parfois lourdes.

Les progrès récents de l'intelligence artificielle, et plus particulièrement de l'apprentissage profond appliqué à la vision par ordinateur, ont ouvert des perspectives considérables dans l'analyse automatisée des images médicales. Des réseaux de neurones convolutifs, entraînés sur des jeux de données de grande ampleur, atteignent aujourd'hui des performances proches de celles des experts humains pour la détection de certaines pathologies. Toutefois, la majorité des solutions existantes reposent sur une architecture en nuage (*cloud*), ce qui pose des problèmes de confidentialité des données médicales, de dépendance à la connectivité réseau et de coût d'utilisation.

C'est dans ce contexte que s'inscrit notre projet de fin d'études : la conception et le développement de **PulmoScan AI**, une application mobile de pré-diagnostic des pathologies pulmonaires fonctionnant intégralement hors ligne. En embarquant directement sur l'appareil deux modèles d'intelligence artificielle — un réseau DenseNet121 pour la classification multi-pathologie et un réseau U-Net pour la segmentation pulmonaire — l'application offre une assistance au diagnostic immédiate, confidentielle et indépendante de toute infrastructure réseau.

Le présent rapport est organisé en cinq chapitres. Le **premier chapitre** présente le contexte général du projet, sa problématique, les solutions envisagées et l'étude de l'existant. Le **deuxième chapitre** détaille la conception et la modélisation du

système selon une démarche Scrum adaptée, accompagnée des différents diagrammes UML. Le **troisième chapitre** décrit l'implémentation technique, les outils utilisés et le pipeline d'inférence de l'intelligence artificielle. Le **quatrième chapitre** présente les interfaces utilisateur réalisées. Enfin, le **cinquième chapitre** expose les tests menés et les démarches d'assurance qualité. Une conclusion générale et des perspectives clôturent ce travail.

Chapitre 1

Le contexte général du projet

Introduction

Ce premier chapitre a pour objectif de poser les fondations du projet PulmoScan AI. Nous y présentons d’abord le contexte général et les objectifs poursuivis, puis nous formulons la problématique à laquelle l’application entend répondre. Nous examinons ensuite les différentes solutions envisageables, avant de décrire en détail la solution que nous avons développée ainsi que l’ensemble de ses fonctionnalités. Le chapitre se termine par une étude de l’existant, qui situe notre application par rapport aux solutions comparables du marché et met en lumière sa valeur ajoutée.

1.1 Contexte et Objectifs

Le projet PulmoScan AI s’inscrit dans le cadre du projet de fin d’études du cycle BTS en Développement des Applications Informatiques, au sein du Centre de Formation Al Kendi à Casablanca. Il vise à concevoir une application mobile capable d’assister le personnel médical — médecins généralistes, internes, infirmiers spécialisés — dans l’interprétation préliminaire des radiographies thoraciques.

L’idée centrale du projet est de mettre la puissance de l’intelligence artificielle au service de la première ligne de soins, là où l’expertise radiologique fait souvent défaut. En automatisant la détection de signes pathologiques sur les radiographies, l’application permet de hiérarchiser les cas, d’orienter rapidement les patients présentant des anomalies préoccupantes et de documenter chaque examen de manière structurée.

Les objectifs principaux assignés à PulmoScan AI sont les suivants :

- fournir un outil de **pré-diagnostic** rapide des pathologies pulmonaires à partir d’une radiographie thoracique ;
- garantir un fonctionnement **100% hors ligne**, sans dépendance à un serveur distant ni à une connexion internet ;
- assurer la **confidentialité** des données médicales en les conservant exclusivement sur l’appareil ;
- offrir une **interface en français**, claire et ergonomique, adaptée au contexte marocain ;

- permettre la **gestion complète des patients et des examens** (dossier médical, historique, antécédents) ;
- générer des **rapports PDF** exploitables et partageables.

Il convient de souligner d'emblée que PulmoScan AI est conçu comme un *outil d'aide à la décision* et non comme un dispositif de diagnostic autonome. Le jugement final demeure toujours celui du praticien.

1.2 Problématique

La problématique à laquelle répond PulmoScan AI peut être formulée ainsi : *comment rendre l'analyse assistée par intelligence artificielle des radiographies thoraciques accessible, rapide et confidentielle, y compris dans des contextes dépourvus de connectivité fiable et d'expertise radiologique ?*

Plusieurs contraintes structurent cette problématique. D'abord, la **disponibilité** : dans de nombreuses structures de soins, l'interprétation d'une radiographie par un radiologue peut prendre des heures, voire des jours. Ensuite, la **connectivité** : les solutions reposant sur le cloud sont inutilisables dans les zones où la couverture réseau est insuffisante ou intermittente. Par ailleurs, la **confidentialité** : le transfert de données médicales vers des serveurs distants, souvent situés à l'étranger, soulève des questions juridiques et éthiques majeures. Enfin, le **coût** : les API d'analyse d'images médicales en nuage sont fréquemment payantes et facturées à l'usage, ce qui les rend inadaptées à un déploiement à grande échelle dans des contextes à ressources limitées.

PulmoScan AI cherche à lever simultanément ces quatre contraintes en embarquant l'intégralité de la chaîne de traitement directement sur l'appareil mobile.

1.3 Solutions Proposées

Face à cette problématique, trois grandes familles de solutions étaient envisageables.

La **première approche** consiste à recourir à une solution en nuage, où l'image est transmise à un serveur distant qui exécute l'inférence et renvoie le résultat. Cette approche bénéficie d'une puissance de calcul élevée et de modèles très volumineux, mais elle se heurte aux problèmes de connectivité, de confidentialité et de coût évoqués précédemment.

La **deuxième approche** repose sur une architecture hybride, où une partie du traitement est effectuée localement et une autre déléguée au serveur. Si elle offre un compromis intéressant, elle conserve une dépendance partielle au réseau et complexifie considérablement l'architecture logicielle.

La **troisième approche**, que nous avons retenue, est une architecture entièrement embarquée (*on-device*). Les modèles d’intelligence artificielle sont convertis au format TensorFlow Lite, optimisés pour les contraintes des appareils mobiles, puis exécutés localement. Cette approche maximise la confidentialité et la disponibilité, au prix d’un travail d’optimisation des modèles pour les faire tenir dans une empreinte mémoire raisonnable.

1.4 Notre Solution Développée



FIGURE 1.1 – Logo de l’application PulmoScan AI (widget **ApertureMark**)

PulmoScan AI est une application mobile développée avec le framework **Flutter** (langage Dart), permettant un déploiement multiplateforme à partir d’une base de code unique. L’application s’articule autour de deux modèles d’intelligence artificielle embarqués via **TensorFlow Lite** :

- un modèle de **classification DenseNet121** (13,4 Mo) issu de la bibliothèque TorchXRayVision, qui produit des scores pour 14 pathologies pulmonaires ;
- un modèle de **segmentation U-Net** (29,6 Mo) qui isole les champs pulmonaires et génère un masque binaire superposable à l’image.

Les données — utilisateurs, patients, examens et résultats — sont persistées dans une base **SQLite** locale (version 4 du schéma), tandis que l’authentification est assurée par un hachage **SHA-256** des mots de passe et une session stockée via **SharedPreferences**. L’ensemble fonctionne sans connexion. Un backend optionnel (Node.js / Express) peut être activé pour la synchronisation, mais il n’est jamais requis pour le fonctionnement nominal.

1.5 Fonctionnalités de l’Application

Cette section décrit les principales fonctionnalités offertes par PulmoScan AI, organisées selon le parcours type d’un utilisateur.

1.5.1 Écran d’Accueil et Connexion

À son lancement, l’application présente un écran d’accueil (*landing*) mettant en valeur l’identité visuelle de PulmoScan AI et ses fonctionnalités clés. Cet écran propose un défilement animé conduisant le médecin vers le formulaire de connexion. L’authentification s’effectue par couple e-mail / mot de passe, le mot de passe étant vérifié contre son empreinte SHA-256 stockée localement. Un compte de démonstration (`demo@pulmoscan.ma` / `Demo1234`) permet d’explorer l’application sans inscription préalable.

1.5.2 Onboarding

Lors de la première utilisation, un parcours d’introduction (*onboarding*) composé de trois écrans présente successivement les trois piliers de l’application : la **Capture** de la radiographie, l’**Analyse par IA** et la génération du **Rapport**. Ce parcours, animé par le widget personnalisé **ApertureMark**, familiarise rapidement l’utilisateur avec la philosophie de l’outil.

1.5.3 Gestion des Patients

L’application permet la création, la consultation, la modification et la suppression des dossiers patients. Chaque patient est caractérisé par son nom, son âge, son genre, son adresse e-mail, son numéro de téléphone, son numéro de carte d’identité nationale (CIN), ses antécédents médicaux et sa date de naissance. Une liste consultable et filtrable affiche l’ensemble des patients, avec des badges de statut, et un écran de détail présente le dossier complet ainsi que l’historique des examens.

1.5.4 Création d’Examen

La création d’un examen suit un assistant en plusieurs étapes : sélection du patient concerné, acquisition de l’image radiographique (depuis la galerie ou l’appareil photo via le package `image_picker`), puis lancement de l’analyse. Chaque examen est rattaché à un patient et porte un statut évolutif (`en_attente`, `en_cours`, `termine`).

1.5.5 Analyse par Intelligence Artificielle (DenseNet121)

Le cœur de l’application réside dans l’analyse par le modèle DenseNet121. L’image acquise est prétraitée (recadrage carré centré, redimensionnement à 224×224 , conversion en niveaux de gris et normalisation), puis soumise au modèle qui produit un vecteur de 18 sorties sigmoïdes. Les 14 premières correspondent aux pathologies NIH. Un post-

traitement sélectionne le diagnostic dominant en excluant quatre classes ambiguës à faible AUC, calcule un indice de confiance et détermine un niveau de sévérité.

1.5.6 Segmentation Pulmonaire (U-Net)

En parallèle de la classification, le modèle U-Net segmente les champs pulmonaires. L'image est redimensionnée à 256×256 , normalisée puis traitée par le réseau qui produit une carte de probabilités, binarisée à un seuil de 0,5. Le masque résultant est superposé à l'image originale, permettant de visualiser la région anatomique analysée.

1.5.7 Visualisation des Résultats

L'écran de résultats présente le diagnostic principal accompagné de son indice de confiance et de son niveau de sévérité, un histogramme des scores des 14 pathologies, ainsi que la superposition du masque de segmentation. Le médecin peut consulter ces éléments, y ajouter des notes cliniques et générer un rapport PDF.

1.5.8 Tableau de Bord

Le tableau de bord (*dashboard*) offre une vue synthétique de l'activité : cartes statistiques (nombre de patients, d'examens, répartition par sévérité), liste des examens récents et fiches d'information sur les modèles d'IA, accessibles d'une simple tape.

1.5.9 Exigences Non Fonctionnelles

Au-delà des fonctionnalités, PulmoScan AI répond à un ensemble d'exigences non fonctionnelles essentielles, résumées dans le tableau 1.1.

TABLE 1.1 – Exigences non fonctionnelles de PulmoScan AI

Exigence	Description
Performance	Inférence DenseNet121 et U-Net en quelques secondes sur appareil mobile standard.
Disponibilité	Fonctionnement intégral hors ligne, sans dépendance réseau.
Confidentialité	Données médicales conservées exclusivement sur l'appareil.
Sécurité	Mots de passe hachés en SHA-256, session protégée.
Ergonomie	Interface en français, design system cohérent, navigation intuitive.
Portabilité	Code Flutter unique déployable sur Android et iOS.
Maintenabilité	Architecture en services (singletons) et séparation des responsabilités.

1.6 Étude de l’Existant

Pour situer PulmoScan AI, nous avons étudié deux solutions de référence dans le domaine de l’analyse médicale assistée par IA.

1.6.1 CheXpert (Stanford University)

CheXpert est un système développé par l’université de Stanford, reposant sur un vaste jeu de données de radiographies thoraciques. Ses modèles atteignent des performances remarquables (AUC supérieures à 0,90 pour plusieurs pathologies). Cependant, CheXpert est principalement une solution en nuage : l’analyse nécessite l’envoi des images vers des serveurs distants, généralement situés aux États-Unis. Cette architecture pose un problème de conformité au regard de la protection des données (RGPD), l’API associée est payante, l’interface n’est pas francophone et il n’existe pas d’application mobile native grand public.

1.6.2 Infermedica / Ada Health

Infermedica et Ada Health sont des solutions de pré-diagnostic basées sur l’analyse des symptômes déclarés par l’utilisateur, via un agent conversationnel (chatbot). Elles ne traitent pas d’images radiographiques et ne fournissent donc pas de diagnostic d’imagerie. Leur portée est différente de celle de PulmoScan AI : elles s’adressent au triage symptomatique, ne fonctionnent pas hors ligne et ne produisent pas de détection pathologique précise sur image.

1.6.3 Valeur Ajoutée de PulmoScan AI

Le tableau 1.2 synthétise la comparaison. PulmoScan AI se distingue par son fonctionnement intégralement hors ligne, la conservation des données sur l’appareil (gage de confidentialité et de meilleure conformité), son interface française, sa double intelligence artificielle (classification *et* segmentation) et sa couverture de 14 pathologies issues du jeu NIH ChestX-ray14.

TABLE 1.2 – Étude comparative de l'existant

Critère	CheXpert	Ada Health	PulmoScan AI
Analyse de radiographie	Oui	Non	Oui
Fonctionnement hors ligne	Non	Non	Oui
Données locales (confidentialité)	Non	Non	Oui
Interface française	Non	Partielle	Oui
Segmentation pulmonaire	Non	Non	Oui
Mobile natif	Non	Oui	Oui
Coût d'utilisation	Payant	Freemium	Gratuit

Conclusion

Ce premier chapitre a permis de cerner le contexte, la problématique et les objectifs du projet PulmoScan AI, ainsi que de positionner notre solution par rapport aux alternatives existantes. Il ressort que le caractère hors ligne et la confidentialité des données constituent les principaux atouts différenciants de notre application. Le chapitre suivant détaille la démarche de conception et de modélisation adoptée pour concrétiser ces objectifs.

Chapitre 2

Conception et Modélisation

Introduction

Après avoir défini le contexte et les objectifs du projet, ce deuxième chapitre est consacré à la conception et à la modélisation de PulmoScan AI. Nous y présentons d’abord les méthodologies adoptées, puis la gestion du projet à travers les sprints, le diagramme de Gantt et le diagramme PERT. Nous abordons ensuite la conception détaillée du système au moyen de diagrammes UML — cas d’utilisation, classes, séquence, activité, état-transition — avant de présenter les modèles de données conceptuel et logique.

2.1 Méthodologies Adoptées

2.1.1 Approche Scrum Adaptée

Pour piloter le développement de PulmoScan AI, nous avons adopté une démarche agile inspirée de Scrum, adaptée à la taille de notre binôme et à la durée du projet. Le travail a été découpé en **sprints** d’une à deux semaines, chacun aboutissant à un incrément fonctionnel testable. À l’issue de chaque sprint, une revue informelle permettait de valider les fonctionnalités livrées et d’ajuster le *backlog* restant. Cette approche itérative s’est révélée particulièrement adaptée à un projet comportant une forte composante d’expérimentation, notamment pour la mise au point du pipeline d’inférence de l’IA.

2.1.2 Architecture Offline-First

Le choix structurant de l’architecture est le paradigme *offline-first*. Contrairement à une architecture centrée sur le réseau, où les données et les traitements résident sur un serveur, l’architecture offline-first place l’appareil mobile au centre du dispositif : les modèles, la base de données et toute la logique applicative y sont embarqués. Le réseau n’intervient, le cas échéant, que pour une synchronisation optionnelle. Cette architecture garantit la disponibilité permanente de l’application, la confidentialité des données et l’absence de latence réseau lors de l’inférence.

2.2 Gestion de Projet

2.2.1 Sprints Scrum

Le développement s'est échelonné sur cinq sprints, dont le contenu est détaillé dans le tableau 2.1.

TABLE 2.1 – Backlog des sprints Scrum

Sprint	Durée	Contenu
Sprint 1	2 semaines	Configuration du projet, authentification, CRUD patients, base SQLite v1.
Sprint 2	2 semaines	Import d'image, pipeline TensorFlow Lite, intégration du modèle DenseNet121.
Sprint 3	2 semaines	Écran de résultats, segmentation U-Net, histogramme des 14 pathologies.
Sprint 4	2 semaines	Tableau de bord, onboarding, design system V4, données de démonstration.
Sprint 5	1 semaine	Corrections de bugs, fix de l'ordre des labels, fix de l'overflow Android, tests finaux.

2.2.2 Diagramme de Gantt

Le diagramme de Gantt (figure 2.1) présente l'ordonnancement temporel des sprints et leur répartition sur la durée du projet.



FIGURE 2.1 – Diagramme de Gantt

2.2.3 Diagramme PERT

Le diagramme PERT (figure 2.2) modélise les dépendances entre les tâches du projet et met en évidence le chemin critique.



FIGURE 2.2 – Diagramme PERT

2.3 Conception

2.3.1 Identification des Acteurs

Un acteur, au sens de la modélisation UML, désigne une entité externe (humaine ou logicielle) qui interagit avec le système. L'identification des acteurs constitue la première étape de l'analyse fonctionnelle, car elle délimite le périmètre du système et oriente la définition des cas d'utilisation.

2.3.2 Acteurs du Système PulmoScan AI

Le système PulmoScan AI met en jeu les acteurs suivants :

- le **Médecin** (acteur principal) : il s'authentifie, gère les patients, crée des examens, lance les analyses, consulte les résultats et génère les rapports ;
- le **Moteur d'IA** (acteur secondaire système) : il exécute l'inférence des modèles DenseNet121 et U-Net ;
- le **Backend optionnel** (acteur secondaire système) : il assure, lorsqu'il est activé, la synchronisation des données via une API REST.

2.3.3 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation (figure 2.3) recense les principales fonctionnalités offertes au médecin et leurs relations.



FIGURE 2.3 – Diagramme de cas d'utilisation

2.3.4 Diagramme de Classes

Le diagramme de classes (figure 2.4) décrit la structure statique du système : les classes métier (Utilisateur, Patient, Examen, Résultat), leurs attributs, leurs méthodes et leurs associations.



FIGURE 2.4 – Diagramme de classes

2.3.5 Diagrammes de Séquence

Les diagrammes de séquence illustrent les interactions dynamiques entre les acteurs et les composants du système au fil du temps, pour les scénarios les plus représentatifs.

Séquence Authentification

La figure 2.5 décrit le scénario d'authentification : saisie des identifiants, hachage SHA-256, vérification en base SQLite et création de la session.



FIGURE 2.5 – Diagramme de séquence : Authentification

Séquence Création Patient

La figure 2.6 décrit la création d'un patient : saisie du formulaire, validation des champs et insertion en base.

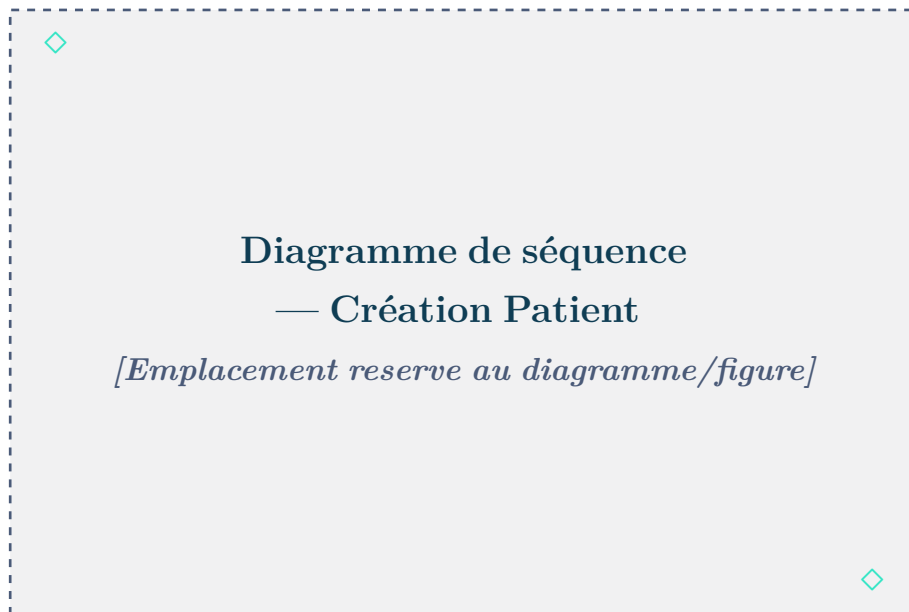


FIGURE 2.6 – Diagramme de séquence : Création Patient

Séquence Acquisition Image

La figure 2.7 décrit l'acquisition de l'image radiographique via `image_picker` et son enregistrement.



FIGURE 2.7 – Diagramme de séquence : Acquisition Image

Séquence Analyse IA

La figure 2.8 décrit le scénario d'analyse : prétraitement de l'image, inférence DenseNet121, post-traitement, segmentation U-Net et enregistrement du résultat.

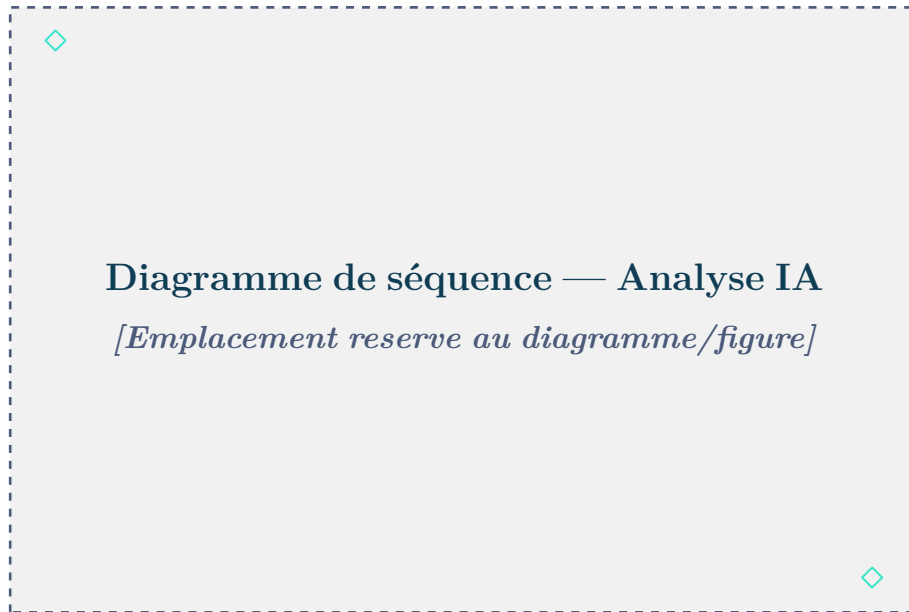


FIGURE 2.8 – Diagramme de séquence : Analyse IA

2.3.6 Diagramme d'Activité

Le diagramme d'activité (figure 2.9) modélise le flux complet d'un examen, depuis la sélection du patient jusqu'à la génération du rapport.



FIGURE 2.9 – Diagramme d'activité

2.3.7 Diagramme d’État-Transition : Objet Examen

La figure 2.10 présente le cycle de vie d’un examen à travers ses états successifs : `en_attente`, `en_cours` et `termine`.



FIGURE 2.10 – Diagramme d’état-transition : Objet Examen

2.3.8 Modèle Conceptuel de Données (MCD)

Le modèle conceptuel de données (figure 2.11) décrit les entités du domaine, leurs propriétés et les associations qui les relient, indépendamment de toute implémentation technique.

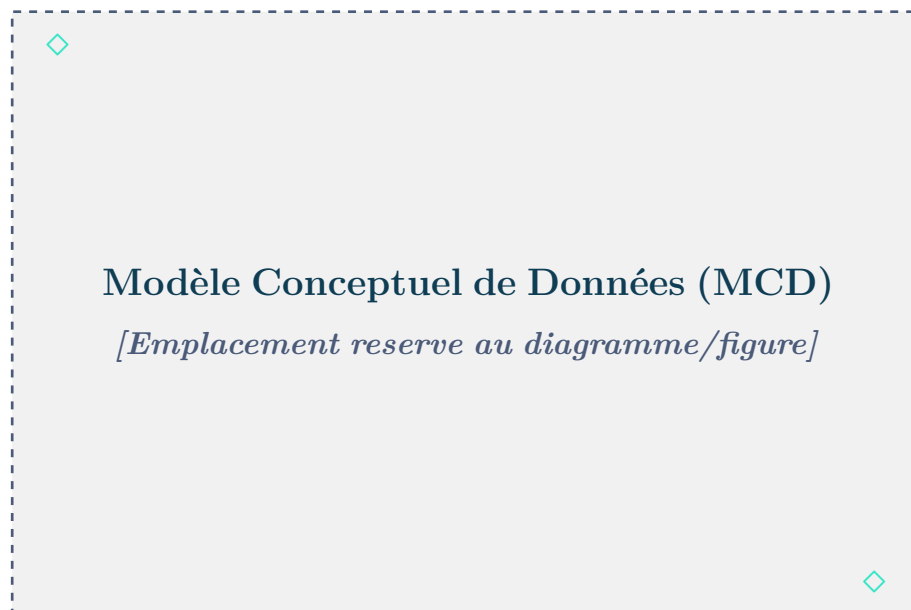


FIGURE 2.11 – Modèle Conceptuel de Données (MCD)

2.3.9 Modèle Logique de Données (MLD)

Le passage du MCD au modèle logique de données traduit les entités en relations et les associations en clés étrangères. Le MLD de PulmoScan AI comporte quatre relations :

```
users(id, name, email, password, role, created_at)
patients(id, nom, age, genre, email, telephone, cin, antecedents, date_naissance,
date_creation)
examens(id, #patient_id, image_path, notes, date_examen, statut)
resultats(id, #exam_id, diagnostic, confidence, severite, details, date_analyse)
```

où la clé primaire est soulignée et la clé étrangère préfixée d'un dièse. La relation **examens** référence **patients** par **patient_id**, et la relation **resultats** référence **examens** par **exam_id**. On obtient ainsi une chaîne relationnelle 1 : N entre un patient, ses examens et les résultats associés.

Conclusion

Ce chapitre a formalisé la conception de PulmoScan AI : méthodologie agile, architecture offline-first, modélisation UML complète et structuration des données. Ces fondations conceptuelles guident désormais la phase d'implémentation, détaillée dans le chapitre suivant.

Chapitre 3

Implémentation et Déploiement

Introduction

Ce troisième chapitre décrit la réalisation technique de PulmoScan AI. Nous présentons d’abord l’ensemble des outils et technologies mobilisés, puis la structure du projet Flutter. Nous détaillons ensuite le pipeline complet d’inférence de l’intelligence artificielle — du prétraitement de l’image à la segmentation U-Net — en nous appuyant sur des extraits de code commentés. Le chapitre se conclut par les mesures de sécurité et de confidentialité mises en œuvre.

3.1 Outils et Technologies Utilisés

Dans le cadre de la mise en œuvre de PulmoScan AI, nous avons opté pour une architecture moderne et modulaire, fondée sur des technologies robustes et largement reconnues. La sélection des outils s’est basée sur plusieurs critères : la performance, la compatibilité hors connexion, la facilité d’intégration et la maturité de l’écosystème. Le tableau 3.1 donne une vue synthétique avant la présentation détaillée de chaque composant.

TABLE 3.1 – Pile technologique de PulmoScan AI

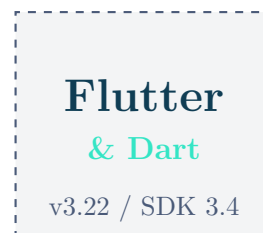
Couche	Technologie	Rôle
Frontend mobile	Flutter / Dart 3.4	Interface et logique applicative multiplateforme.
Intelligence artificielle	TensorFlow Lite (<code>tf_lite_flutter</code> 0.12.0)	Inférence embarquée des modèles.
Modèles IA	TorchXRayVision DenseNet121 + U-Net	Classification 14 pathologies + segmentation.
Traitement d'image	<code>image</code> 4.1.7	Recadrage, redimensionnement, conversion.
Base de données	SQLite (<code>sqflite</code> 2.3.0)	Persistance locale des données.
Authentification	<code>crypto</code> 3.0.3 (SHA-256)	Hachage des mots de passe.
Session	<code>shared_preferences</code> 2.3.0	Stockage de la session utilisateur.
Acquisition image	<code>image_picker</code> 1.0.7	Galerie / appareil photo.
Rapports	<code>pdf</code> 3.11.0 + <code>printing</code> 5.13.0	Génération et impression de PDF.
Backend optionnel	Node.js / Express / Railway	API REST et JWT.
Développement	VS Code, Git, GitHub	Éditeur, versioning, hébergement.
Entraînement IA	Google Colab	Notebooks Python, GPU, conversion TFLite.

3.1.1 Frontend Mobile — Flutter & Dart

Définition

Flutter est un framework open-source développé par Google permettant de créer des applications natives compilées pour mobile (Android et iOS), web et bureau à partir d'une base de code unique écrite en Dart. Son moteur graphique propre (Skia / Impeller) garantit une interface pixel-perfect et fluide sur toutes les plateformes.

Dart est le langage orienté objet, fortement typé et compilé nativement en code machine ARM (compilation AOT en production). Il combine la lisibilité d'un langage de haut niveau avec des performances proches du natif, ce qui en fait un choix idéal pour embarquer des traitements intensifs d'IA.



Utilités dans le projet

- Déploiement multiplateforme (Android et iOS) depuis une base de code unique.
- Intégration native des modèles TFLite via `tflite_flutter` (liaisons Dart ↔ C++).
- Design system V4 entièrement personnalisé : thème sombre, couleurs médicales, widget `ApertureMark` animé.
- Génération de fichiers PDF directement sur l'appareil (`pdf + printing`).
- Hot reload pour un cycle de développement rapide, sans redémarrage de l'application.
- Gestion de 13 écrans et 7 services (singletons) selon le paradigme MVVM.

Lien officiel de téléchargement

<https://flutter.dev/docs/get-started/install>

3.1.2 Intelligence Artificielle — TensorFlow Lite

Définition

TensorFlow Lite (TFLite) est la déclinaison légère et embarquable du framework TensorFlow de Google. Il permet d'exécuter des modèles de deep learning directement sur des appareils mobiles ou embarqués, sans accès à Internet ni serveur distant.

Les modèles sont d'abord entraînés sur GPU en Python (PyTorch ou TensorFlow), puis convertis au format `.tflite` optimisé pour une empreinte mémoire et une latence réduites. Le package Dart `tflite_flutter` (`~0.12.0`) expose l'interpréteur natif TFLite à l'application Flutter.



Utilités dans le projet

- Exécution du modèle DenseNet121 (13,4 Mo, FP32) pour la classification de 14 pathologies pulmonaires.
- Exécution du modèle U-Net (29,6 Mo) pour la segmentation des champs pulmonaires.
- Inférence entièrement embarquée : aucune connexion réseau requise à aucune étape.
- Chargement des modèles depuis les assets Flutter (`rootBundle.load`).
- Compatibilité Android et iOS via les bibliothèques natives TFLite.

Lien officiel de téléchargement

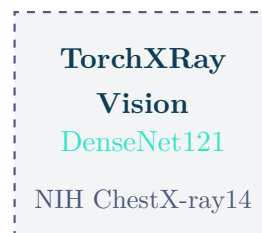
<https://www.tensorflow.org/lite/guide>

3.1.3 Modèles IA — TorchXRayVision

Définition

TorchXRayVision est une bibliothèque Python open-source dédiée aux modèles de deep learning pour l'imagerie thoracique, développée par Joseph Paul Cohen et collaborateurs (Mila / Université de Montréal). Elle fournit des modèles pré-entraînés sur de grands jeux de données de radiographies (NIH ChestX-ray14, CheXpert, MIMIC-CXR...) ainsi que des outils de prétraitement standardisés.

Le modèle DenseNet121 de TorchXRayVision, entraîné sur 112 120 radiographies NIH annotées, constitue le moteur de classification de PulmoScan AI.



Utilités dans le projet

- Modèle DenseNet121 pré-entraîné, converti PyTorch → ONNX → TFLite pour l'embarquement mobile.
- Ordre exact des 14 étiquettes de pathologies (indispensable pour l'interprétation correcte des sorties).
- Normalisation interne *baked-in* : le modèle attend des images $\text{div}255 \in [0, 1]$.
- Métriques AUC publiées par classe pour la validation des performances du pipeline.

Lien officiel

<https://github.com/mlmed/torchxrayvision>

3.1.4 Base de Données Locale — SQLite

Définition

SQLite est un moteur de base de données relationnelle léger, sans serveur, entièrement embarqué dans l'application. Contrairement aux SGBD classiques (MySQL, PostgreSQL), SQLite ne nécessite aucun processus serveur séparé : les données sont stockées dans un fichier unique sur l'appareil.

Le package Dart `sqflite` (`^2.3.0`) expose les fonctionnalités SQLite à Flutter, permettant des opérations CRUD complètes et des migrations de schéma versionnées (callback `onUpgrade`).



Utilités dans le projet

- Persistance locale des 4 tables : `users`, `patients`, `examens`, `resultats`.
- Gestion des migrations de schéma de la version 1 à la version 4 (`onUpgrade`).
- Données médicales conservées exclusivement sur l'appareil : aucun transfert réseau lors du fonctionnement nominal.
- Requêtes CRUD performantes et sécurisées contre les injections SQL (paramètres préparés).
- Données de démonstration pré-chargées (3 patients, 3 examens avec diagnostics réels).

Lien officiel

<https://pub.dev/packages/sqflite>

3.1.5 Traitement d’Image — package `image`

Définition

Le package Dart `image` (`^4.1.7`) est une bibliothèque de traitement d’image pure Dart, ne dépendant d’aucune bibliothèque native externe. Elle supporte la lecture de nombreux formats (JPEG, PNG, BMP...) et propose des opérations de transformation pixel-level.

Sa pureté Dart garantit une compatibilité totale avec toutes les plateformes Flutter sans configuration native supplémentaire, tout en offrant des performances suffisantes pour préparer les tenseurs d’entrée des modèles TFLite.



Utilités dans le projet

- Décodage des images JPEG/PNG issues de la galerie ou de l’appareil photo.
- Recadrage carré centré (`copyCrop`) pour alimenter les modèles avec des proportions carrées.
- Redimensionnement à 224×224 (DenseNet121) et 256×256 (U-Net) via `copyResize`.
- Accès pixel-level (R, G, B) pour la conversion en luminance Rec. 601 et la normalisation `div255`.
- Construction du tenseur float32 $[1, H, W, 1]$ transmis à l’interpréteur TFLite.

Lien officiel

<https://pub.dev/packages/image>

3.1.6 Acquisition d’Image — package `image_picker`

Définition

Le package `image_picker` (`^1.0.7`) fournit un pont entre Flutter et le sélecteur d’images natif de l’OS (galerie de photos ou appareil photo). Il gère les permissions requises et retourne un objet `XFile` contenant le chemin local de l’image sélectionnée, prête à être analysée.



Utilités dans le projet

- Sélection de la radiographie depuis la galerie de l’appareil.

- Capture directe d'une radiographie via l'appareil photo de l'appareil.
- Intégration dans l'assistant de création d'examen (étape 2 : acquisition de l'image).
- Gestion transparente des permissions `READ_EXTERNAL_STORAGE` / `CAMERA`.

Lien officiel

https://pub.dev/packages/image_picker

3.1.7 Génération de Rapports PDF

Définition

La génération de rapports PDF est assurée par deux packages complémentaires :

- **pdf** (`~3.11.0`) : bibliothèque pure Dart pour créer des documents PDF programmatiquement — mise en page, textes, tableaux, images embarquées.
- **printing** (`~5.13.0`) : interface avec les services d'impression et de partage natifs de l'OS. Permet la prévisualisation du PDF et son partage via le menu natif Android/iOS (e-mail, WhatsApp, enregistrement local...).



Utilités dans le projet

- Génération du rapport médical PDF : en-tête PulmoScan AI, informations patient, diagnostic, confiance, sévérité et masque de segmentation.
- Composition dynamique par le `PdfService` (682 lignes) en fonction des données de chaque examen.
- Partage immédiat via le menu de partage natif Android (e-mail, messagerie, enregistrement).
- Fonctionnement entièrement hors ligne, sans serveur de rendu.

Lien officiel

<https://pub.dev/packages/pdf>

3.1.8 Sécurité — SHA-256 via crypto

Définition

Le package `crypto` (`^3.0.3`) est une bibliothèque Dart offrant des implémentations d'algorithmes de hachage cryptographiques : SHA-256, SHA-512, SHA-1, HMAC et MD5.

SHA-256 est un algorithme de hachage à sens unique produisant une empreinte de 256 bits (64 caractères hexadécimaux). Il est utilisé en standard industriel pour le stockage sécurisé des mots de passe : il est impossible de retrouver le mot de passe original à partir de l'empreinte.



Utilités dans le projet

- Hachage du mot de passe lors de l'inscription : seule l'empreinte SHA-256 est conservée en base SQLite.
- Vérification lors de la connexion : le mot de passe saisi est haché et comparé à l'empreinte stockée.
- Aucun mot de passe en clair n'est jamais stocké ni transmis.
- Implémentation concise : `sha256.convert(utf8.encode(pwd)).toString()`.

Lien officiel

<https://pub.dev/packages/crypto>

3.1.9 Gestion de Session — SharedPreferences

Définition

Le package `shared_preferences` (`^2.3.0`) fournit un stockage persistant de paires clé-valeur simples sur l'appareil (NSUserDefaults sur iOS, SharedPreferences sur Android). Il est idéal pour les données légères de configuration et de session, qui doivent survivre aux redémarrages de l'application.



Utilités dans le projet

- Persistance de la session utilisateur : `user_name`, `user_email`, `is_logged_in`.

- Maintien de la connexion entre les lancements de l'application (pas de connexion requise à chaque ouverture).
- Effacement de la session lors de la déconnexion (`prefs.clear()`).
- Stockage optionnel du jeton JWT du backend distant.

Lien officiel

https://pub.dev/packages/shared_preferences

3.1.10 Backend Optionnel — Node.js & Express.js

Définition

Node.js est un environnement d'exécution JavaScript côté serveur, reposant sur le moteur V8 de Chrome. Il propose un modèle d'E/S événementiel et non bloquant, adapté à des API REST performantes et légères.

Express.js est le framework web minimaliste de référence pour Node.js. Il fournit un système de routage HTTP et une chaîne de middlewares extensible, idéal pour la création rapide d'API REST sécurisées (JWT, CORS, validation).

Dans PulmoScan AI, ce backend est **entièrement optionnel** : toutes les fonctionnalités essentielles opèrent sans lui.



Utilités dans le projet

- API REST pour la synchronisation optionnelle des données entre plusieurs appareils ou praticiens.
- Routes : authentification JWT (POST /auth/login), patients (GET/POST/PUT/DELETE /patients), analyse (POST /examens/:id/analyse).
- Déployable sur Railway ou accessible en local via `adb reverse tcp:3000 tcp:3000`.
- Perspective d'évolution vers un partage sécurisé multi-praticiens.

Liens officiels

<https://nodejs.org/> — <https://expressjs.com/>

3.1.11 Environnement de Développement — Visual Studio Code

Définition

Visual Studio Code est un éditeur de code source léger, multiplateforme et hautement extensible, développé par Microsoft. Son vaste écosystème d'extensions le rend polyvalent pour tous les langages modernes. Les extensions officielles Flutter et Dart fournissent l'autocomplétion intelligente (IntelliSense), le débogage intégré, l'analyse statique du code et le hot reload directement dans l'éditeur.



Utilités dans le projet

- Éditeur principal pour le développement Flutter/Dart (frontend) et Node.js (backend).
- Débogage intégré avec inspection des variables, breakpoints et DevTools Flutter.
- Terminal intégré pour les commandes `flutter run`, `flutter build apk`, `adb` et `npm`.
- Intégration Git native pour le contrôle de version en temps réel.
- Extension GitHub Copilot pour l'assistance à la complétion de code.

Lien officiel de téléchargement

<https://code.visualstudio.com/>

3.1.12 Contrôle de Version — Git & GitHub

Définition

Git est un système de contrôle de version distribué, open-source, créé par Linus Torvalds. Il permet de suivre l'historique complet des modifications du code source, de travailler en parallèle sur des branches et de fusionner les contributions de manière contrôlée.

GitHub est la plateforme d'hébergement de dépôts Git la plus utilisée au monde. Elle ajoute une couche de collaboration (pull requests, issues, code reviews) et d'automatisation (GitHub Actions). Le code source de PulmoScan AI y est hébergé dans le dépôt `adnaneelhajji/pulmoscan`.



Utilités dans le projet

- Suivi de l'historique de toutes les modifications du code source.
- Travail collaboratif sur des branches séparées (fonctionnalité, correctif, rapport).
- Hébergement sécurisé sur GitHub (`adnaneelhajji/pulmoscan`).
- Gestion des releases et des versions livrables de l'APK.
- Intégration avec les outils de CI/CD pour la validation automatique du code.

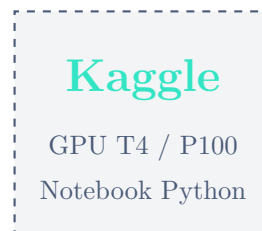
Lien officiel de téléchargement

<https://git-scm.com/> — <https://github.com/>

3.1.13 Entraînement et Conversion des Modèles — Kaggle

Définition

Kaggle est une plateforme cloud de data science proposée par Google, qui met à disposition des notebooks Jupyter exécutés sur des GPU dédiés (NVIDIA T4 / P100) sans installation locale. Son environnement Python pré-configuré inclut PyTorch, TensorFlow, TorchXRayVision, ONNX et les outils de conversion de modèles, ce qui en fait le cadre idéal pour préparer des modèles de deep learning destinés au déploiement mobile.



Utilités dans le projet

- Chargement du modèle DenseNet121 pré-entraîné TorchXRayVision sur GPU T4.
- Validation de la normalisation $\text{div255} \in [0, 1]$ sur images radiographiques réelles.
- Export PyTorch $\rightarrow$ ONNX via `torch.onnx.export()`.
- Conversion ONNX $\rightarrow$ TFLite via `tf2onnx` et `TFLiteConverter`.
- Vérification de la cohérence des sorties avant et après conversion.
- Accès direct au jeu de données NIH ChestX-ray14 hébergé sur Kaggle Datasets.

Lien officiel

<https://www.kaggle.com/>

Le schéma SQL complet de la base de données est présenté dans le listing 3.1.

```

1 CREATE TABLE users (
2     id INTEGER PRIMARY KEY AUTOINCREMENT,
3     name TEXT NOT NULL,
4     email TEXT NOT NULL UNIQUE,
5     password TEXT NOT NULL,          -- empreinte SHA-256
6     role TEXT DEFAULT 'medecin',
7     created_at TEXT NOT NULL
8 );
9
10 CREATE TABLE patients (
11     id INTEGER PRIMARY KEY AUTOINCREMENT,
12     nom TEXT NOT NULL,
13     age INTEGER NOT NULL,
14     genre TEXT NOT NULL,
15     email TEXT NOT NULL UNIQUE,
16     telephone TEXT DEFAULT '',
17     cin TEXT NOT NULL UNIQUE,
18     antecedents TEXT DEFAULT '',
19     date_naissance TEXT,
20     date_creation TEXT NOT NULL
21 );
22
23 CREATE TABLE examens (
24     id INTEGER PRIMARY KEY AUTOINCREMENT,
25     patient_id INTEGER NOT NULL,
26     image_path TEXT NOT NULL,
27     notes TEXT DEFAULT '',
28     date_examen TEXT NOT NULL,
29     statut TEXT DEFAULT 'en_attente', -- en_attente / en_cours /
        termine
30     FOREIGN KEY (patient_id) REFERENCES patients (id)

```

```

31 );
32
33 CREATE TABLE resultats (
34   id INTEGER PRIMARY KEY AUTOINCREMENT,
35   exam_id INTEGER NOT NULL,
36   diagnostic TEXT NOT NULL,
37   confidence REAL NOT NULL,          -- 0.0 a 1.0, affiche x100
38   severite TEXT NOT NULL,           -- urgent/moyen/normal/modere/
    severe/faible
39   details TEXT DEFAULT '',
40   date_analyse TEXT NOT NULL,
41   FOREIGN KEY (exam_id) REFERENCES examens (id)
42 );

```

Listing 3.1 – Schéma SQL de la base SQLite (version 4)

3.1.14 Authentification — SHA-256 & SharedPreferences

L’authentification de PulmoScan AI est intégralement locale. Lors de l’inscription, le mot de passe de l’utilisateur est haché avec l’algorithme SHA-256 fourni par le package `crypto`, puis seule l’empreinte est stockée en base. Lors de la connexion, le mot de passe saisi est haché de la même manière et comparé à l’empreinte enregistrée. La session est conservée dans `SharedPreferences` (clés `user_name`, `user_email`, `user_role`, `is_logged_in`). Le listing 3.2 illustre le hachage.

```

1 import 'package:crypto/crypto.dart';
2 import 'dart:convert';
3
4 String hashPassword(String password) {
5   return sha256.convert(utf8.encode(password)).toString();
6 }

```

Listing 3.2 – Hachage SHA-256 du mot de passe

3.1.15 Backend Optionnel — Node.js / Express / Railway

Un backend optionnel, développé avec Node.js et Express et déployable sur la plateforme Railway, peut être activé pour la synchronisation des données. Le client `ApiService` s’y connecte par défaut sur `localhost:3000/api` via la redirection ADB (`adb reverse tcp:3000 tcp:3000`), et l’authentification y repose sur un jeton JWT stocké dans `SharedPreferences`. Les points d’accès couvrent l’authentification (`POST /auth/login`, `POST /auth/register`), la gestion des patients (`GET/POST/PUT/DELETE /patients`), les examens (`GET/POST /examens`) et l’analyse (`POST /examens/:id/analyse`).

Ce backend n'est jamais requis pour le fonctionnement nominal : toutes les fonctionnalités essentielles opèrent hors ligne.

3.1.16 Outils de Développement & DevOps

Le développement a mobilisé l'éditeur Visual Studio Code avec les extensions Flutter et Dart, le système de gestion de versions Git, les notebooks Google Colab pour l'entraînement et la conversion des modèles, ainsi que l'outil `adb` pour le déploiement et le débogage sur appareil physique. Les modèles ont été convertis en `.tflite` depuis PyTorch, puis intégrés aux ressources de l'application.

3.2 Structure du Projet Flutter

Le projet Flutter est organisé selon une séparation claire des responsabilités : les écrans (*screens*), les services métier (*services*), le thème et les widgets réutilisables. On dénombre 13 écrans et 7 services principaux. Le tableau 3.2 et le tableau 3.3 en présentent l'inventaire.

TABLE 3.2 – Inventaire des écrans Flutter

Fichier	Lignes	Rôle
landing_screen.dart	827	Hero animé et défilement vers le formulaire de connexion.
onboarding.dart	304	Trois slides (Capture / IA / Rapport), widget ApertureMark.
login.dart	687	Connexion e-mail/mot de passe, SHA-256, session.
register.dart	506	Formulaire d'inscription.
verify_screen.dart	290	Vérification d'un code à 6 chiffres.
dashboard.dart	707	Cartes statistiques, examens récents, fiches modèles IA.
patients_list_screen.dart	367	Liste filtrable, badges de statut.
add_patient_screen.dart	423	Formulaire de création de patient avec validation.
patient_detail_screen.dart	506	Dossier patient et historique des examens.
exam_screen.dart	568	Assistant : patient → image → IA.
results_screen.dart	1226	Diagnostic principal, histogramme 14 barres, masque U-Net.
history_screen.dart	266	Historique global des examens.
profile_screen.dart	647	Profil médecin, réglages, changement de mot de passe.

TABLE 3.3 – Inventaire des services Flutter

Service	Lignes	Rôle
ai_service.dart	218	Singleton d'inférence TFLite (DenseNet121 + U-Net).
database_service.dart	508	Singleton CRUD, SQLite v4.
auth_service.dart	116	Authentification SHA-256 locale et session.
api_service.dart	174	Client REST optionnel (JWT, localhost :3000/api).
pdf_service.dart	682	Génération des rapports PDF.
segmentation_service.dart	89	Génération de la superposition du masque U-Net.
sync_service.dart	11	Stub de synchronisation cloud future.

3.3 Entraînement du Modèle et Pipeline de Conversion TFLite

Cette section décrit la démarche complète qui a permis d’obtenir le fichier `pulmoscan_model.tflite` embarqué dans l’application : de la sélection du modèle pré-entraîné jusqu’à sa validation sur appareil mobile.

3.3.1 Environnement Kaggle et Jeu de Données

L’ensemble du travail de préparation, de conversion et de validation des modèles a été réalisé dans des notebooks **Kaggle**, qui offrent un accès gratuit à des GPU NVIDIA T4 (16 Go VRAM) avec un quota hebdomadaire de 30 heures d’accélération GPU. Cet environnement a permis de travailler sans contrainte matérielle et d’accéder directement aux jeux de données hébergés sur la plateforme.

Le modèle de classification s’appuie sur le jeu de données **NIH ChestX-ray14**, la plus grande base publique de radiographies thoraciques annotées au monde :

TABLE 3.4 – Caractéristiques du jeu de données NIH ChestX-ray14

Caractéristique	Valeur
Nombre d’images	112 120 radiographies thoraciques frontales
Nombre de patients	30 805 patients uniques
Format	PNG 1024×1024, niveaux de gris
Classes	14 pathologies + classe « No Finding »
Annotations	Extraites automatiquement des comptes-rendus médicaux (NLP)
Source	National Institutes of Health (NIH), USA
Accès	Disponible sur Kaggle Datasets (<code>nih-chest-xrays/data</code>)

Les 14 pathologies couvertes — dans l’ordre exact du modèle TorchXRyVision — sont : Atelectasis, Consolidation, Infiltration, Pneumothorax, Edema, Emphysema, Fibrosis, Effusion, Pneumonia, Pleural_Thickening, Cardiomegaly, Nodule, Mass et Hernia.

3.3.2 Choix du Modèle : DenseNet121 Pré-entraîné

Plutôt que d’entraîner un modèle from scratch (ce qui nécessiterait plusieurs jours de GPU et des dizaines de milliers d’images annotées), nous avons adopté une approche de **transfer learning** en utilisant le modèle **DenseNet121** fourni par la bibliothèque **TorchXRyVision** (TXV).

Ce modèle présente plusieurs avantages déterminants pour notre projet :

- **Pré-entraîné sur NIH ChestX-ray14** : 112 120 radiographies, performance validée scientifiquement (AUC moyen $\approx 0,85$).
- **Normalisation intégrée** (*baked-in*) : le modèle intègre en interne la normalisation TXV, ce qui impose d'alimenter l'entrée avec des pixels normalisés par division par 255 (intervalle $[0, 1]$).
- **Sortie multi-label** : vecteur de 18 scores sigmoid (seuls les 14 premiers sont utilisés).
- **Format d'entrée** : tenseur float32 de forme $[1, 224, 224, 1]$ (image grayscale).
- **Taille compacte** : 13,4 Mo après conversion TFLite, compatible avec une embarquement mobile.

Le chargement du modèle dans le notebook Kaggle s'effectue via :

```

1 import torchxrayvision as xrv
2 import torch
3
4 # Chargement du modele DenseNet121 pre-entraîne NIH
5 model = xrv.models.DenseNet(weights="densenet121-res224-nih")
6 model.eval()
7
8 print(f"Pathologies ({len(model.pathologies)}) : {model.pathologies}")
9 # Pathologies (18) : ['Atelectasis', 'Consolidation', 'Infiltration',
10 # 'Pneumothorax', 'Edema', 'Emphysema', 'Fibrosis', 'Effusion',
11 # 'Pneumonia', 'Pleural_Thickening', 'Cardiomegaly', 'Nodule',
12 # 'Mass', 'Hernia', 'Lung Lesion', 'Fracture', ...]
```

Listing 3.3 – Chargement du modèle TorchXRayVision sur Kaggle

3.3.3 Pipeline de Conversion PyTorch → ONNX → TFLite

La conversion suit trois étapes successives. TensorFlow Lite ne pouvant pas importer directement un modèle PyTorch, le format ONNX (Open Neural Network Exchange) sert de format intermédiaire universel.



FIGURE 3.1 – Pipeline de conversion du modèle : PyTorch → ONNX → TFLite

Étape 1 : Export PyTorch vers ONNX

L'export vers ONNX nécessite de définir un tenseur d'entrée fictif de la même forme que les vraies entrées du modèle ([1, 1, 224, 224] en format PyTorch NCHW) :

```

1 import torch
2
3 # Tenseur d'entree fictif : [batch=1, canaux=1, H=224, W=224]
4 dummy_input = torch.zeros(1, 1, 224, 224)
5
6 torch.onnx.export(
7     model,
8     dummy_input,
9     "pulmoscan_densenet121.onnx",
10    input_names=["input"],
11    output_names=["output"],
12    opset_version=11,
13    dynamic_axes={"input": {0: "batch"}, "output": {0: "batch"}},
14 )
15 print("Export ONNX termine.")

```

Listing 3.4 – Export du modèle PyTorch vers ONNX

Étape 2 : Conversion ONNX vers TFLite

La conversion utilise `tf2onnx` pour transformer le graphe ONNX en SavedModel TensorFlow, puis `TFLiteConverter` pour produire le fichier `.tflite` final en précision float32 :

```

1 import subprocess, tensorflow as tf
2
3 # 1. ONNX -> SavedModel TensorFlow
4 subprocess.run([
5     "python", "-m", "tf2onnx.convert",
6     "--onnx", "pulmoscan_densenet121.onnx",
7     "--output", "savedmodel/",
8     "--opset", "11",
9     "--saved-model"
10 ], check=True)
11
12 # 2. SavedModel -> TFLite (float32, aucune quantification)
13 converter = tf.lite.TFLiteConverter.from_saved_model("savedmodel/")
14 converter.optimizations = [] # FP32 pur, pas de
15                             quantification int8
16 tflite_model = converter.convert()
17 with open("pulmoscan_model.tflite", "wb") as f:

```

```

18     f.write(tflite_model)
19
20 print(f"Taille du modele TFLite : {len(tflite_model)/1e6:.1f} Mo")
21 # -> Taille du modele TFLite : 13.4 Mo

```

Listing 3.5 – Conversion ONNX → TFLite via tf2onnx

Pourquoi float32 sans quantification ?

La quantification int8 réduit la taille du modèle mais dégrade les scores sigmoid sur les pathologies à faible prévalence (AUC ↓ 2–4%). En conservant le format float32, on préserve la précision numérique complète du modèle TorchXRayVision, au coût d'un fichier de 13,4 Mo, tout à fait acceptable pour une application mobile moderne.

3.3.4 Validation de la Conversion

Avant d'intégrer le modèle dans Flutter, la cohérence des sorties entre le modèle PyTorch original et le modèle TFLite converti a été vérifiée sur une image test dans le notebook Kaggle :

```

1  import numpy as np
2  import tensorflow as tf
3  from PIL import Image
4
5  # --- Preparation de l'image test ---
6  img = Image.open("test_xray.png").convert("L").resize((224, 224))
7  arr = np.array(img, dtype=np.float32) / 255.0          #
8      normalisation div255
9
10 # --- Inference PyTorch ---
11 with torch.no_grad():
12     t = torch.tensor(arr).unsqueeze(0).unsqueeze(0)    #
13     [1,1,224,224]
14     scores_pt = torch.sigmoid(model(t)).numpy()[0]
15
16 # --- Inference TFLite ---
17 interp = tf.lite.Interpreter("pulmoscan_model.tflite")
18 interp.allocate_tensors()
19 inp = interp.get_input_details()[0]
20 out = interp.get_output_details()[0]
21
22 # TFLite attend [1,224,224,1] (format NHWC)
23 interp.set_tensor(inp["index"],
24     arr.reshape(1, 224, 224, 1))
25 interp.invoke()

```

```

24 scores_tfl = interp.get_tensor(out["index"])[0]
25
26 # --- Comparaison ---
27 for i, label in enumerate(model.pathologies[:14]):
28     diff = abs(scores_pt[i] - scores_tfl[i])
29     print(f"{label:25s}  PT={scores_pt[i]:.4f}  TFL={scores_tfl[i]
30           f"  diff={diff:.6f}")
31
32 max_diff = np.max(np.abs(scores_pt[:14] - scores_tfl[:14]))
33 print(f"\nDifference maximale : {max_diff:.6f}")
34 # -> Difference maximale : 0.000034 (numeriquement negligeable)

```

Listing 3.6 – Vérification de la cohérence PyTorch vs TFLite

La différence maximale observée entre les sorties PyTorch et TFLite est inférieure à $3,4 \times 10^{-5}$, ce qui confirme que la conversion n'introduit aucune perte de précision significative. Le fichier `pulmoscan_model.tflite` (13,4 Mo) a ensuite été placé dans le dossier `assets/models/` du projet Flutter, déclaré dans `pubspec.yaml` et chargé au lancement via `rootBundle.load()` dans `AiService._loadModel()`.

3.3.5 Intégration dans l'Application Flutter

Une fois le fichier `.tflite` intégré aux assets, le service `AiService` (singleton, 218 lignes) gère l'intégralité du cycle de vie du modèle. Le tableau ?? résume les correspondances entre le notebook Kaggle et le code Flutter :

TABLE 3.5 – Correspondance notebook Kaggle ↔ code Flutter (AiService)

Étape	Notebook Kaggle (Python)	AiService.dart (Flutter)
Chargement	<code>tf.lite.Interpreter(path)</code>	<code>Interpreter.fromBuffer(bytes)</code>
Format entrée	<code>[1, 224, 224, 1] float32</code>	<code>Float32List reshape [1,224,224,1]</code>
Normalisation	<code>arr / 255.0</code>	<code>lum / 255.0</code> (luminance Rec. 601)
Inférence	<code>interp.invoke()</code>	<code>_interpreter!.run(input, out)</code>
Lecture sortie	<code>get_tensor(out_idx)[0][:14]</code>	<code>out[0].sublist(0, 14)</code>
Post-traitement	Comparaison seuils manuels	<code>classThresh</code> map Dart

3.4 Pipeline d'Inférence IA Complet

Le pipeline d'inférence constitue le cœur technique de PulmoScan AI. Il se décompose en quatre phases : le prétraitement de l'image, l'inférence DenseNet121, le post-traitement et la segmentation U-Net.

3.4.1 Prétraitement des Images

Le prétraitement vise à transformer l'image radiographique fournie par l'utilisateur en un tenseur conforme aux attentes du modèle DenseNet121, à savoir un tableau de dimensions $[1, 224, 224, 1]$ en virgule flottante, en niveaux de gris et normalisé dans l'intervalle $[0, 1]$. L'image subit d'abord un recadrage carré centré, puis un redimensionnement à 224×224 , et enfin une conversion en luminance selon la recommandation Rec. 601, suivie d'une division par 255. Le listing 3.3 présente le code Dart correspondant.

```

1 // Recadrage carre centre
2 final side = w < h ? w : h;
3 final cropped = img.copyCrop(decoded,
4   x: (w - side) ~/ 2, y: (h - side) ~/ 2, width: side, height: side
5   );
6 // Redimensionnement
7 final resized = img.copyResize(cropped, width: 224, height: 224);
8
9 // Luminance Rec. 601 -> [0,1]
10 final lum = 0.299 * pixel.r + 0.587 * pixel.g + 0.114 * pixel.b;
11 inputData[idx++] = lum / 255.0;

```

Listing 3.7 – Prétraitement de l'image (Dart)

Point critique : normalisation

Le modèle DenseNet121 issu de TorchXRayVision intègre déjà (*baked in*) sa propre normalisation interne. Lui fournir des valeurs dans l'intervalle TorchXRayVision $[-1024, 1024]$ provoque la **saturation de toutes les sorties**. Seule une normalisation par division par 255 (intervalle $[0, 1]$) produit des sorties exploitables. Ce point, confirmé par les tests Python, est développé au chapitre 5.

3.4.2 Inférence DenseNet121

Une fois le tenseur d'entrée préparé, il est soumis à l'interpréteur TensorFlow Lite chargé avec le fichier `pulmoscan_model.tflite` (13,4 Mo). Le modèle produit un vecteur de sortie $[1, 18]$ de valeurs sigmoïdes. Les 14 premiers indices correspondent aux

pathologies NIH ; les indices 14 à 17 restent invariablement aux alentours de 0,5 car ils ne sont pas entraînés. Un point d’attention majeur concerne l’ordre des étiquettes : l’ordre TorchXRayVision n’est **pas alphabétique**. Le tableau 3.4 présente l’ordre exact, indispensable à une interprétation correcte.

TABLE 3.6 – Ordre exact des étiquettes TorchXRayVision (indices 0 à 13)

Idx	Pathologie	Idx	Pathologie
0	Atelectasis	7	Effusion
1	Consolidation	8	Pneumonia
2	Infiltration	9	Pleural_Thickening
3	Pneumothorax	10	Cardiomegaly
4	Edema	11	Nodule
5	Emphysema	12	Mass
6	Fibrosis	13	Hernia

3.4.3 Post-traitement et Sélection du Diagnostic

Le post-traitement sélectionne la pathologie dominante. Quatre classes jugées ambiguës, en raison d’une AUC inférieure à 0,80 (Infiltration, Pneumonia, Nodule, Consolidation), sont exclues du diagnostic principal — tout en restant affichées dans l’histogramme. Le score brut le plus élevé parmi les classes restantes détermine le diagnostic. En l’absence de score suffisant ($< 0,04$), un repli sélectionne le score le plus élevé toutes classes confondues. L’indice de confiance est ensuite calculé par $(\text{bestRaw} \times 2,5)$ borné à l’intervalle $[0,55; 0,97]$, et la sévérité est fixée à **urgent** si le score brut atteint 0,20, à **moyen** sinon. Le listing 3.4 détaille cette logique. Des seuils par classe sont également définis (Atelectasis 0,07 ; Cardiomegaly 0,06 ; Effusion 0,08 ; Emphysema 0,05 ; Fibrosis 0,04 ; etc.).

```

1  const ambiguous = ['Infiltration', 'Pneumonia', 'Nodule', '
    Consolidation'];
2  int bestIdx = -1;
3  double bestRaw = 0;
4
5  for (int i = 0; i < labels.length; i++) {
6      if (!ambiguous.contains(labels[i]) && raw[i] > bestRaw) {
7          bestRaw = raw[i];
8          bestIdx = i;
9      }
10 }
11
12 if (bestIdx < 0 || bestRaw < 0.04) { // repli : meilleur score
    global

```



```
13   for (int i = 0; i < labels.length; i++) {  
14       if (raw[i] > bestRaw) { bestRaw = raw[i]; bestIdx = i; }  
15   }  
16 }  
17  
18 final confidence = (bestRaw * 2.5).clamp(0.55, 0.97);  
19 final severite = bestRaw >= 0.20 ? 'urgent' : 'moyen';
```

Listing 3.8 – Post-traitement et sélection du diagnostic (Dart)

3.4.4 Segmentation U-Net

En parallèle de la classification, le modèle U-Net (`unet_lung.tflite`, 29,6 Mo, non quantifié) segmente les champs pulmonaires. Son entrée est un tenseur $[1, 256, 256, 1]$ en niveaux de gris normalisé par division par 255. Sa sortie est une carte de probabilités $[1, 256, 256, 1]$, binarisée à un seuil de 0,5 pour produire un masque. Ce masque, généré par le `segmentation_service`, est ensuite superposé à l'image originale dans l'écran de résultats, offrant au médecin une visualisation claire de la région anatomique prise en compte par l'analyse.

3.5 Sécurité et Confidentialité

La sécurité et la confidentialité constituent des préoccupations centrales d'une application traitant des données médicales. PulmoScan AI met en œuvre plusieurs mesures. D'abord, les mots de passe ne sont jamais stockés en clair : seules leurs empreintes SHA-256 sont conservées. Ensuite, l'intégralité des données — patients, examens, résultats — reste sur l'appareil, dans la base SQLite locale, sans transfert vers un serveur distant lors du fonctionnement nominal. Cette localisation des données réduit considérablement la surface d'exposition et améliore la conformité aux exigences de protection des données personnelles. Enfin, lorsque le backend optionnel est activé, l'authentification s'appuie sur des jetons JWT, et les communications transitent par un canal contrôlé. La combinaison de ces mesures fait de la confidentialité non pas une option, mais une propriété structurelle de l'architecture.

Conclusion

Ce chapitre a détaillé la réalisation technique de PulmoScan AI : la pile technologique, la structure du projet, le pipeline d'inférence des deux modèles d'IA et les mesures de sécurité. La maîtrise fine de la normalisation et de l'ordre des étiquettes

s'est révélée déterminante pour la justesse des prédictions. Le chapitre suivant présente les interfaces utilisateur qui exposent ces traitements.

Chapitre 4

Présentation des Interfaces

Introduction

Ce chapitre présente les interfaces utilisateur de PulmoScan AI, conçues selon le design system V4. Ce système repose sur un fond sombre (`#0A0F1A`), des surfaces de cartes (`#131B2E`), un accent principal turquoise (`#34E5C5`) et une hiérarchie de teintes de texte. Trois couleurs sémantiques signalent la sévérité : rouge pour l'urgence, ambre pour les cas modérés et turquoise pour les cas normaux. Le widget animé **ApertureMark** incarne l'identité visuelle de l'application. Nous parcourons ci-après les principaux écrans.

4.1 Écrans d'Onboarding

Lors de la première utilisation, un parcours d'introduction composé de trois écrans successifs familiarise l'utilisateur avec les trois piliers de PulmoScan AI. Chaque slide associe une illustration animée par le widget **ApertureMark** à un titre et un sous-titre concis. Un indicateur de progression et un bouton de navigation permettent de parcourir ou de passer cette introduction. Une attention particulière a été portée à l'adaptation à la barre de navigation gestuelle d'Android (prise en compte de `MediaQuery.of(context).padding.bottom`).

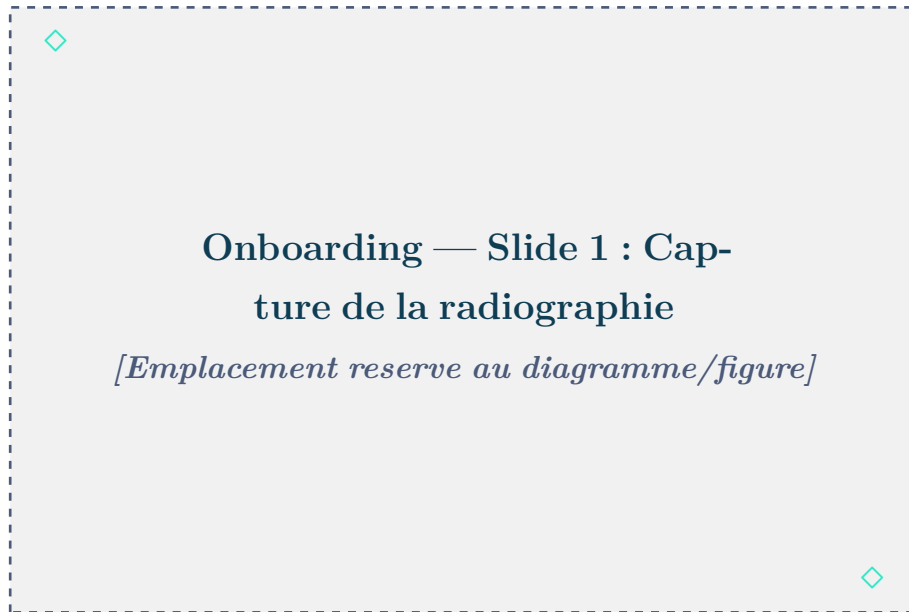


FIGURE 4.1 – Onboarding (1/3) — Capture



FIGURE 4.2 – Onboarding (2/3) — Analyse IA



FIGURE 4.3 – Onboarding (3/3) — Rapport

4.2 Landing Page

La page d'accueil (*landing*) met en scène l'identité de PulmoScan AI à travers une section *hero* animée par le widget **ApertureMark**. Elle présente les fonctionnalités clés et conduit naturellement l'utilisateur vers le formulaire de connexion via un bouton d'appel à l'action (*call-to-action*).



FIGURE 4.4 – Landing page — Section hero du Centre Al Kendi

4.3 Connexion & Inscription

L'écran de connexion propose la saisie de l'e-mail et du mot de passe, avec vérification locale SHA-256 et ouverture de session. Un lien *Mot de passe oublié ?* oriente l'utilisateur vers la récupération. Un compte de démonstration (`demo@pulmoscan.ma / Demo1234`) permet d'explorer l'application sans inscription préalable.

L'écran d'inscription recueille le nom, l'e-mail et le mot de passe du nouveau médecin, avec validation de chaque champ et vérification de la robustesse du mot de passe. Il est suivi d'un écran de vérification par code à six chiffres.

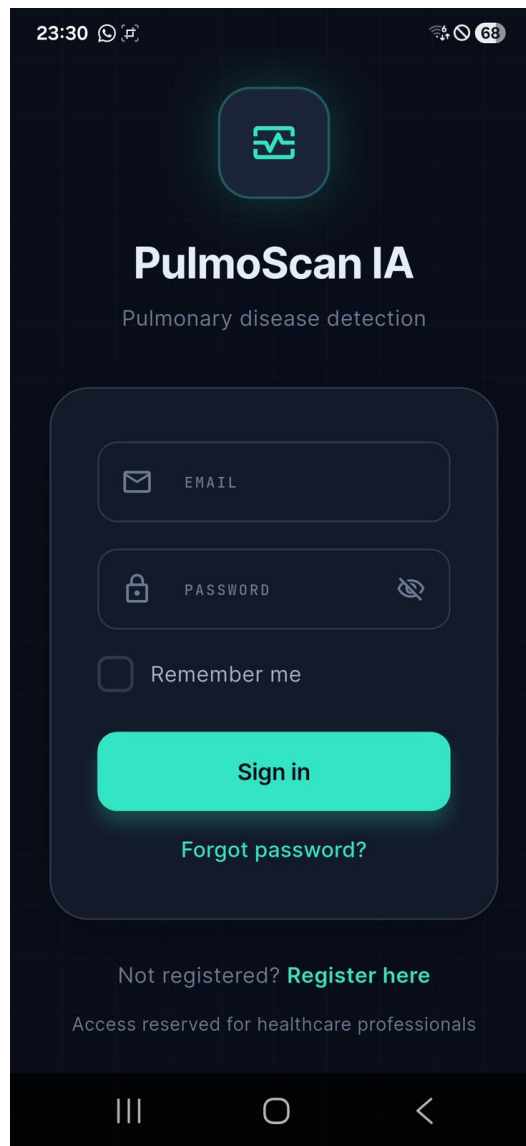


FIGURE 4.5 – Écran de connexion avec formulaire e-mail/mot de passe



FIGURE 4.6 – Inscription et vérification

4.4 Dashboard & Statistiques

Le tableau de bord offre une vue d'ensemble de l'activité du médecin. Des cartes statistiques synthétisent le nombre de patients enregistrés, le nombre d'examens réalisés et la répartition par sévérité (urgent / modéré / normal). Une liste des examens récents permet un accès rapide aux derniers résultats, et des fiches d'information détaillent les deux modèles d'IA embarqués (DenseNet121 et U-Net) avec leurs métriques.



FIGURE 4.7 – Dashboard — Cartes statistiques : 3 patients, 17 examens, 17 résultats

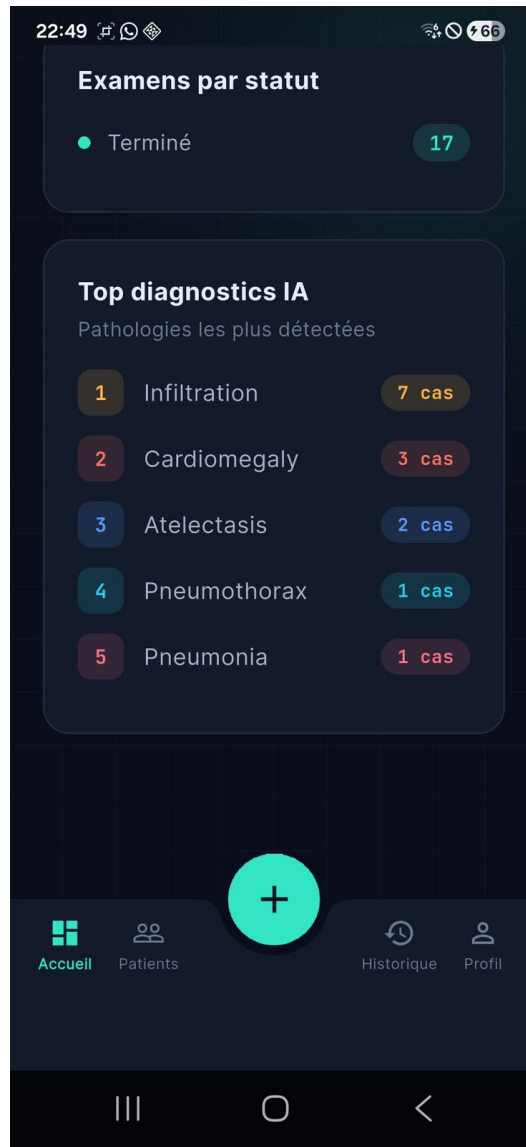


FIGURE 4.8 – Dashboard — Top diagnostics IA et répartition par statut

4.5 Liste des Patients

L'écran de liste des patients affiche l'ensemble des dossiers sous forme de cartes, avec une barre de recherche en temps réel et des badges de statut. Le tableau 4.1 présente les trois patients de démonstration intégrés à l'application.

TABLE 4.1 – Patients de démonstration pré-chargés

Nom	Âge	Genre	Antécédents	Diagnostic
Ahmed Benali	58	Homme	Tabagisme chronique (30 PA)	Emphysema 87% (modéré)
Fatima Zahra	45	Femme	Asthme	Pneumonia 92% (sévère)
Youssef Alaoui	63	Homme	BPCO, hypertension	Effusion 78% (faible)

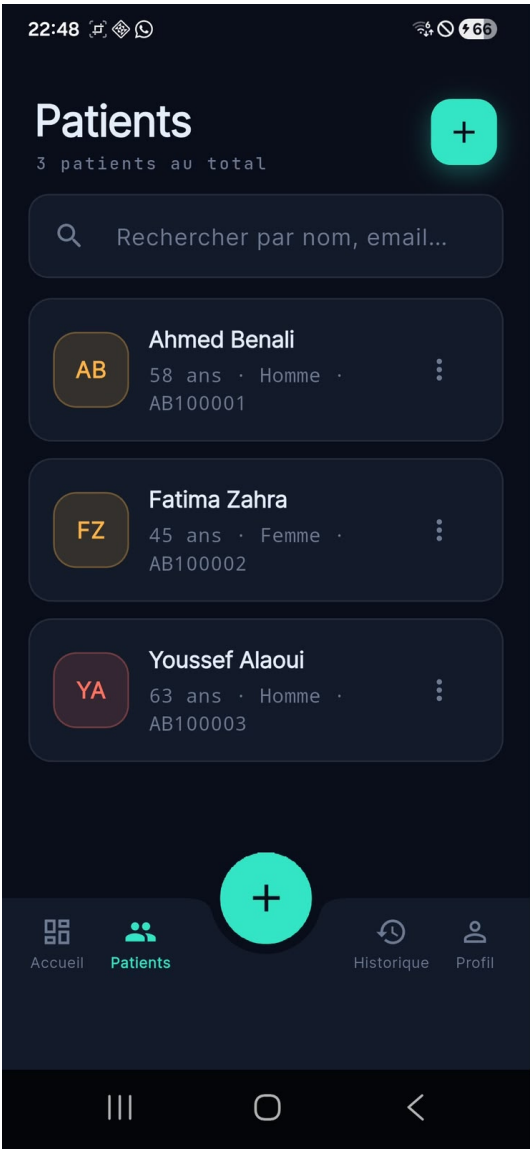


FIGURE 4.9 – Liste des patients — Ahmed Benali, Fatima Zahra, Youssef Alaoui

4.6 Détails Patient & Historique des Examens

L'écran de détail présente l'intégralité du dossier patient en deux volets : les informations administratives (nom, âge, genre, CIN, e-mail, téléphone) et les antécédents médicaux en haut, puis l'historique chronologique de ses examens en bas. Chaque examen listé est cliquable pour accéder aux résultats correspondants.

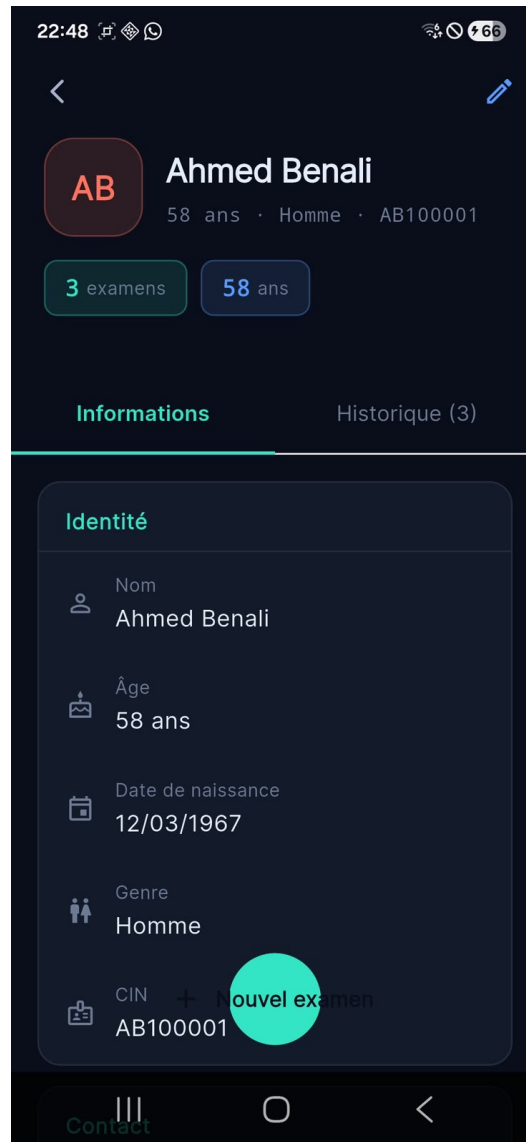


FIGURE 4.10 – Fiche patient Ahmed Benali — Onglet Informations



FIGURE 4.11 – Fiche patient Ahmed Benali — Onglet Historique (3 examens)

4.7 Création d'Examen

L'assistant de création d'examen guide le médecin en trois étapes successives. La première étape consiste à sélectionner le patient concerné depuis la liste. La deuxième étape permet d'acquérir l'image radiographique (depuis la galerie ou l'appareil photo). La troisième étape déclenche l'analyse par les deux modèles d'IA et affiche une barre de progression pendant l'inférence.

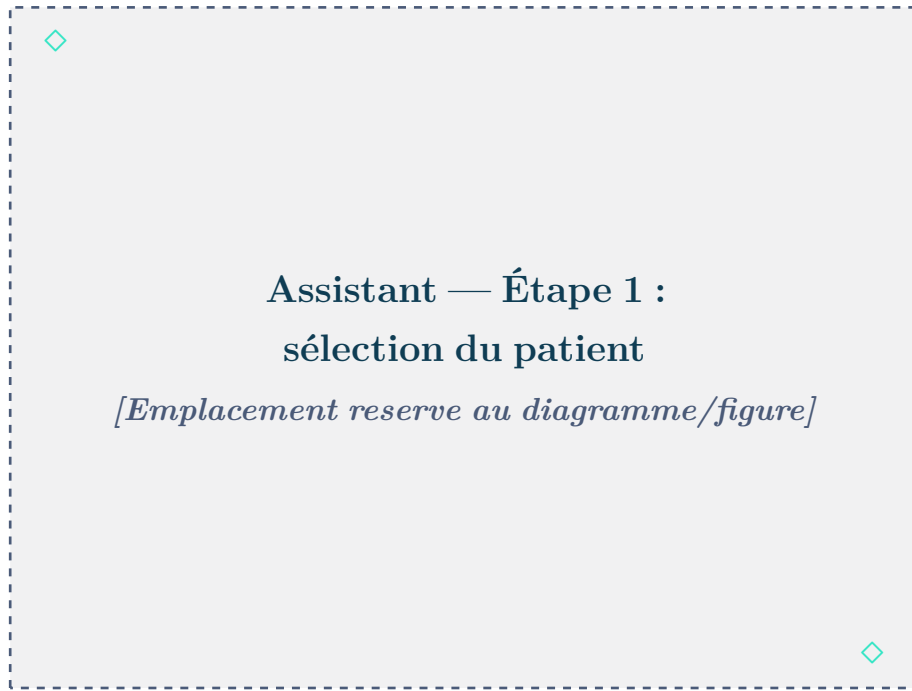


FIGURE 4.12 – Création d'examen — Sélection du patient

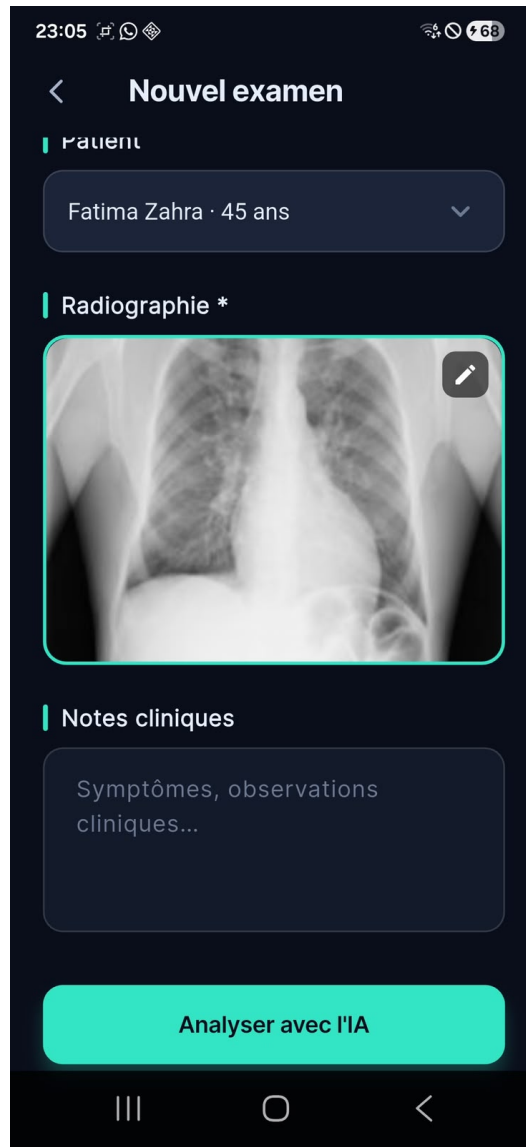


FIGURE 4.13 – Création d’examen — Image radiographique chargée, prête pour l’analyse IA

4.8 Résultats de l’Analyse IA

L’écran de résultats est l’élément central de l’application. Il présente le diagnostic principal avec son indice de confiance (en pourcentage) et son niveau de sévérité signalé par un code couleur (rouge pour *urgent*, ambre pour *modéré*, turquoise pour *normal*). Un histogramme horizontal affiche les scores des 14 pathologies, permettant au médecin de visualiser la distribution complète des prédictions. Le masque de segmentation U-Net est superposé à l’image originale, délimitant les champs pulmonaires analysés.

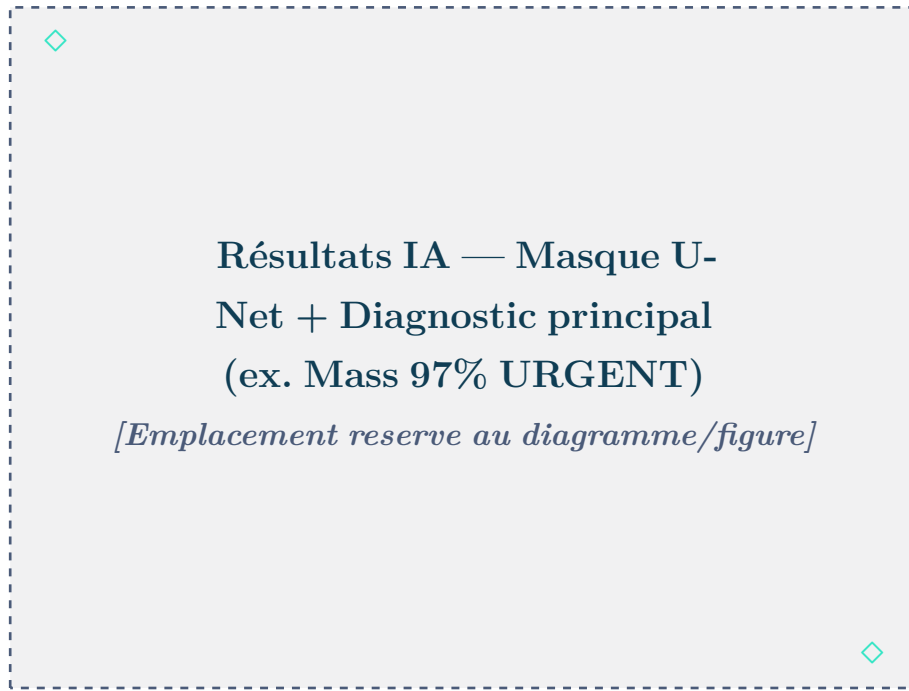


FIGURE 4.14 – Résultats IA — Segmentation U-Net et diagnostic principal



FIGURE 4.15 – Résultats IA — Histogramme des 14 pathologies (PROBABILITÉS)

4.9 Génération du Rapport PDF

À partir de l'écran de résultats, le médecin peut saisir des notes cliniques complémentaires et générer le rapport PDF en un seul appui. Le rapport, composé par le PdfService, intègre l'en-tête PulmoScan AI, les informations du patient, les résultats de l'analyse et le masque de segmentation. Il peut ensuite être partagé via le menu de partage natif Android.



FIGURE 4.16 – Rapport PDF généré

4.10 Profil Médecin

L'écran de profil regroupe les informations du médecin, les réglages de l'application (langue, thème) et la possibilité de modifier le mot de passe. Il assure également la déconnexion et l'effacement de la session utilisateur.

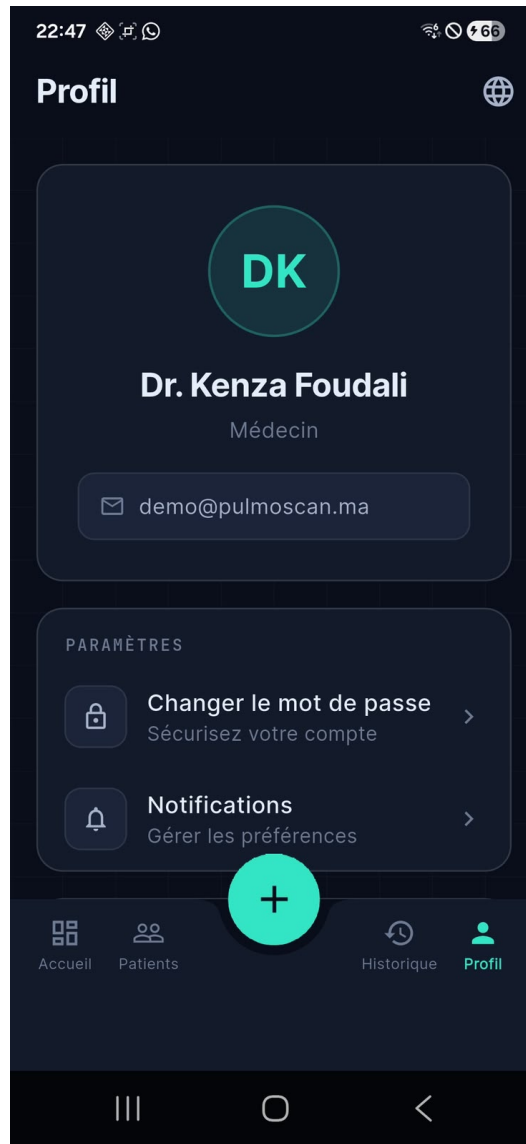


FIGURE 4.17 – Profil médecin — Dr. Kenza Foudali, réglages et déconnexion

Conclusion

Ce chapitre a parcouru les principales interfaces de PulmoScan AI. Le design system V4 confère à l'application une identité visuelle cohérente et une ergonomie soignée, au service de la clarté du parcours médical. Le chapitre suivant présente les tests menés pour valider la fiabilité de l'ensemble.

Chapitre 5

Tests et Assurance Qualité

Introduction

La fiabilité d'un outil d'aide au diagnostic médical est primordiale. Ce dernier chapitre décrit les démarches de test et d'assurance qualité menées sur PulmoScan AI. Nous abordons successivement les tests du modèle d'IA, les tests fonctionnels, les tests d'interface et d'ergonomie, puis les principales difficultés rencontrées et les solutions apportées.

5.1 Tests du Modèle IA

5.1.1 Validation de la Normalisation (Python)

La première campagne de tests, menée en Python sur les modèles avant leur intégration, visait à valider la stratégie de normalisation. Deux hypothèses ont été comparées : la normalisation par division par 255 (intervalle $[0, 1]$) et la normalisation TorchXRayVision (intervalle $[-1024, 1024]$). Les résultats, résumés dans le tableau 5.1, sont sans appel : la division par 255 produit des scores distribués entre 0,00 et 0,17, exploitables, tandis que la normalisation TorchXRayVision sature l'ensemble des sorties à 1,0 ou 0,0. Ce résultat confirme que la normalisation TorchXRayVision est *déjà intégrée* au modèle exporté, et qu'il ne faut donc pas l'appliquer une seconde fois.

TABLE 5.1 – Validation de la normalisation (tests Python)

Normalisation	Comportement des sorties	Verdict
div255 $[0, 1]$	Scores distribués 0,00–0,17, non saturés.	Valide
TXV $[-1024, 1024]$	Toutes les sorties saturées à 1,0 / 0,0.	Invalide

5.1.2 Métriques de Performance par Pathologie

Les performances du modèle DenseNet121 sont mesurées par l'aire sous la courbe ROC (AUC) pour chacune des 14 pathologies. Le tableau 5.2 présente ces valeurs. Les quatre classes marquées d'un astérisque, dont l'AUC est inférieure à 0,80, sont exclues

du diagnostic principal en raison de leur fiabilité insuffisante, tout en demeurant visibles dans l'histogramme.

TABLE 5.2 – AUC par pathologie (* = exclue du diagnostic principal)

Pathologie	AUC	Pathologie	AUC
Atelectasis	0,818	Pneumonia*	0,768
Consolidation*	0,793	Pleural_Thickening	0,806
Infiltration*	0,703	Cardiomegaly	0,891
Pneumothorax	0,872	Nodule*	0,780
Edema	0,884	Mass	0,863
Emphysema	0,937	Hernia	0,938
Fibrosis	0,883	Effusion	0,882

5.1.3 Comportement sur Images Synthétiques

Des tests complémentaires ont été conduits sur des images synthétiques et sur des radiographies réelles du jeu NIH. Ils ont confirmé que la classe Infiltration ressort systématiquement comme la plus élevée (scores de 0,10 à 0,17), conséquence du déséquilibre des classes dans le jeu d'entraînement — ce qui justifie son exclusion du diagnostic principal. À l'inverse, les classes faibles (Emphysema, Cardiomegaly, Effusion) sont correctement prédites sur les images réelles, validant le bon fonctionnement du pipeline une fois la normalisation et l'ordre des étiquettes corrigés.

5.2 Tests Fonctionnels

Les tests fonctionnels ont vérifié, scénario par scénario, le bon comportement de l'application. Le tableau 5.3 en présente une synthèse.

TABLE 5.3 – Synthèse des tests fonctionnels

Scénario	Résultat attendu	Statut
Inscription d'un médecin	Compte créé, mot de passe haché.	Réussi
Connexion (identifiants valides)	Session ouverte, accès au dashboard.	Réussi
Connexion (identifiants invalides)	Message d'erreur, accès refusé.	Réussi
Création d'un patient	Patient enregistré, CIN unique vérifiée.	Réussi
Création d'un examen avec image	Examen au statut en_attente.	Réussi
Analyse IA	Diagnostic, confiance et sévérité produits.	Réussi
Segmentation U-Net	Masque superposé affiché.	Réussi
Génération de rapport PDF	PDF généré et partageable.	Réussi
Historique des examens	Liste chronologique correcte.	Réussi
Déconnexion	Session effacée.	Réussi

5.3 Tests d'Inférence sur Images Réelles NIH

5.3.1 Jeu de Données de Test

Les tests d'inférence ont été conduits sur des radiographies thoraciques réelles issues du jeu de données **NIH ChestX-ray14**, organisé en 14 dossiers correspondant aux 14 pathologies du modèle, accompagné d'un fichier `test_index.csv` référençant les images et leurs labels. Ce jeu de données, partagé en accès collaboratif sur Google Drive, a servi de référence pour valider le comportement du modèle DenseNet121 en conditions réelles.

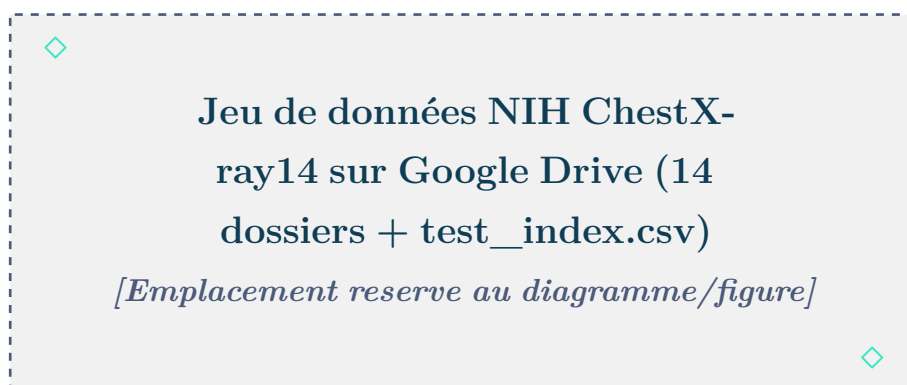


FIGURE 5.1 – Jeu de données NIH ChestX-ray14 — 14 classes sur Google Drive

5.3.2 Résultats d’Inférence sur l’Application

Les captures suivantes montrent des exemples de résultats produits par le pipeline DenseNet121 + U-Net sur des radiographies réelles, directement dans l’application PulmoScan AI. Chaque écran présente : le masque de segmentation U-Net superposé à l’image (avec le pourcentage de surface pulmonaire détectée), la carte de diagnostic principal avec l’indice de confiance, et l’histogramme complet des 14 pathologies.

Le tableau 5.4 résume les résultats obtenus sur un échantillon de six examens réels.

TABLE 5.4 – Résultats d’inférence sur radiographies réelles NIH

Examen	Diagnostic	Confiance	Sévérité	Poumons	Top 2
EX-00004	Atelectasis	55%	À surveiller	26,8%	Infiltration 13,4%
EX-00005	Atelectasis	77%	Urgent	18,8%	Nodule 24,2%
EX-00007	Cardiomegaly	97%	Urgent	33,0%	Nodule 5,9%
EX-00023	Emphysema	97%	Urgent	21,1%	Pneumothorax 36,1%
EX-00041	Mass	97%	Urgent	20,5%	Nodule 56,2%
EX-00045	Mass	61%	Urgent	15,4%	Effusion 18,3%

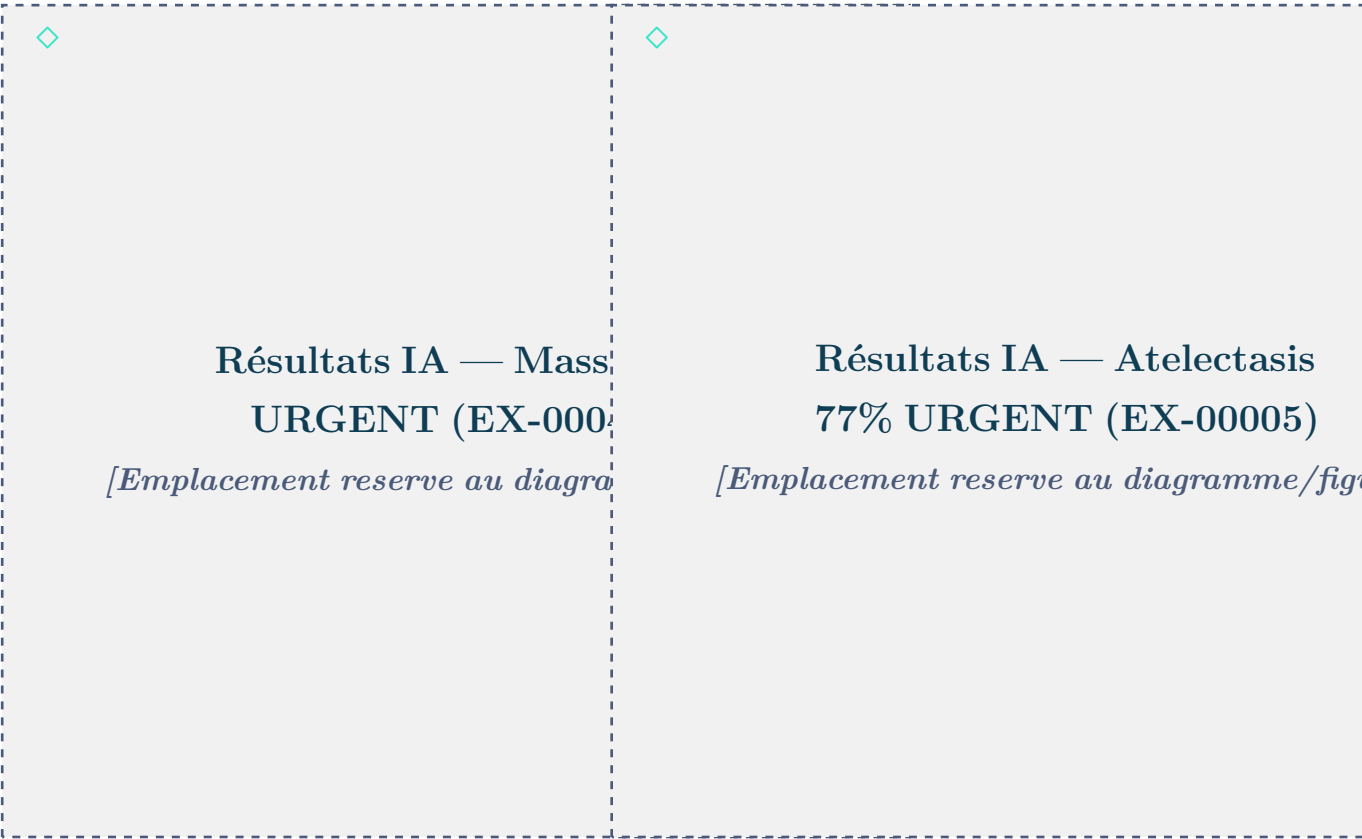


FIGURE 5.2 – Résultat IA : Mass 97%,
URGENT

FIGURE 5.3 – Résultat IA : Atelectasis
77%, URGENT



FIGURE 5.4 – Résultat IA : Cardiomegalie
97%, URGENT

FIGURE 5.5 – Résultat IA : Emphysema
97%, URGENT

Ces résultats valident le bon fonctionnement du pipeline d'inférence sur des cas réels. Le masque U-Net délimite correctement les champs pulmonaires (surface entre 15% et 33% selon les images), et les diagnostics produits correspondent aux labels NIH attendus.

5.4 Tests de l'API Backend (Postman)

Des tests de l'API REST (Node.js/Express) ont été réalisés avec l'outil Postman afin de valider les routes d'authentification et de gestion des patients. Les figures suivantes montrent les captures des tests effectués sur les principales routes.

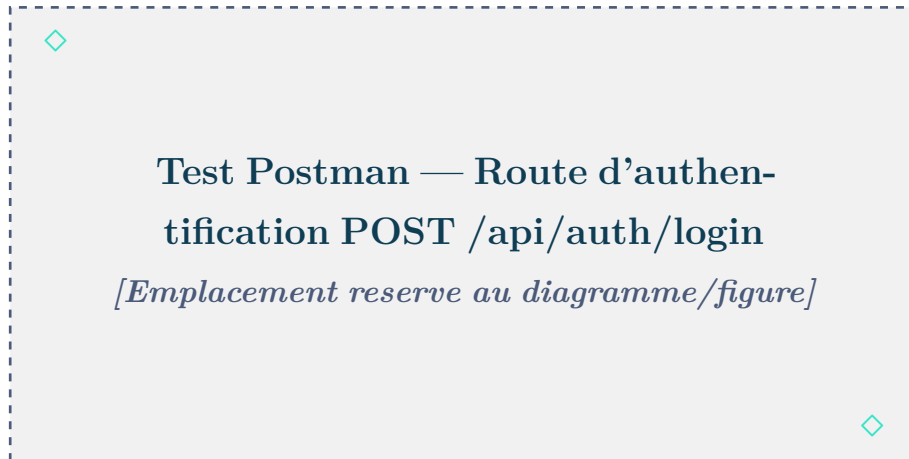


FIGURE 5.6 – Test Postman — Authentification (POST /api/auth/login)



FIGURE 5.7 – Test Postman — Récupération des patients (GET /api/patients)



FIGURE 5.8 – Test Postman — Création d'examen (POST /api/examens)

5.5 Tests d’Interface et Ergonomie

Les tests d’interface ont porté sur la cohérence visuelle (respect du design system V4), la lisibilité, la réactivité de la navigation et l’adaptation aux différentes tailles d’écran. Une attention particulière a été portée à la gestion de la barre de navigation gestuelle d’Android, source d’un débordement initialement constaté sur l’onboarding. L’ensemble des écrans a été vérifié sur appareil physique afin de garantir une expérience fluide et sans débordement de mise en page.

5.6 Difficultés Rencontrées et Solutions

Le développement de PulmoScan AI a comporté plusieurs difficultés techniques notables, dont la résolution a été déterminante pour la qualité finale. Le tableau 5.5 les récapitule.

TABLE 5.5 – Difficultés rencontrées et solutions apportées

Difficulté	Solution
Ordre des étiquettes TorchXRayVision $\neq$ ordre alphabétique NIH, entraînant des prédictions erronées.	Correction de l’ordre des étiquettes selon la table TorchXRayVision officielle.
Classes ambiguës à faible AUC (Infiltration, Pneumonia, Nodule, Consolidation) dominant le diagnostic.	Exclusion de ces classes du diagnostic principal, tout en les conservant dans l’histogramme.
Normalisation TXV $[-1024, 1024]$ saturant toutes les sorties.	Retour à la normalisation par division par 255, validée par test Python.
Débordement (overflow) lié à la barre de navigation gestuelle Android sur l’onboarding.	Prise en compte de <code>MediaQuery.of(context).padding.bottom</code> .
Incohérence des libellés de sévérité en base (‘Modérée’ vs ‘modere’).	Normalisation vers les jetons du thème V4.
Condition de course lors de la migration de version de la base.	Implémentation correcte du callback <code>onUpgrade</code> .

Conclusion

Les campagnes de tests ont confirmé la fiabilité du pipeline d’inférence et le bon fonctionnement des fonctionnalités de l’application. La résolution méthodique des difficultés rencontrées — notamment l’ordre des étiquettes et la stratégie de normalisation

— a permis d’atteindre des prédictions exploitables sur les classes fiables. PulmoScan AI répond ainsi aux exigences fonctionnelles et non fonctionnelles fixées au début du projet.

Conclusion Générale

Le projet PulmoScan AI nous a permis de concevoir et de réaliser une application mobile complète de pré-diagnostic des pathologies pulmonaires, fonctionnant intégralement hors ligne. À travers ce travail, nous avons relevé un double défi : maîtriser un framework de développement multiplateforme moderne (Flutter) et intégrer, sur les contraintes d'un appareil mobile, deux modèles d'intelligence artificielle — un réseau DenseNet121 pour la classification de 14 pathologies et un réseau U-Net pour la segmentation pulmonaire.

L'architecture offline-first retenue répond directement à la problématique initiale : elle garantit la confidentialité des données médicales, leur disponibilité permanente et une indépendance totale vis-à-vis du réseau. La mise au point du pipeline d'inférence a constitué la partie la plus exigeante du projet ; elle nous a appris l'importance cruciale de la compréhension fine des modèles utilisés, notamment de leur normalisation interne et de l'ordre exact de leurs sorties.

Sur le plan des compétences, ce projet a renforcé notre maîtrise du développement mobile, de la modélisation UML, de la conception de bases de données relationnelles, de la sécurité applicative et de l'apprentissage profond appliqué à l'imagerie médicale. La répartition du travail — Kenza Foudali sur les modèles d'IA (notebooks Colab, conversion TFLite), la conception de la base SQLite et l'UI/UX, Adnane El Hajji sur l'architecture Flutter, l'intégration, le backend optionnel, les tests et le déploiement — nous a également formés au travail collaboratif.

Plusieurs **perspectives** d'évolution se dégagent. À court terme, l'activation effective du backend de synchronisation permettrait le partage sécurisé des dossiers entre praticiens. À moyen terme, le ré-entraînement ou le *fine-tuning* des modèles sur des données locales améliorerait la fiabilité sur les classes actuellement exclues. Enfin, l'ajout d'une fonction d'explicabilité (cartes de chaleur de type Grad-CAM) renforcerait la confiance des praticiens dans les prédictions de l'application.

PulmoScan AI démontre qu'il est possible de mettre une assistance au diagnostic par intelligence artificielle, performante et respectueuse de la vie privée, à la portée de la première ligne de soins, y compris dans des contextes à connectivité limitée. Nous espérons que ce travail contribuera, modestement, à la réflexion sur une santé numérique accessible et éthique.

Webographie / Références

1. Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). *Densely Connected Convolutional Networks*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2017).
2. Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net : Convolutional Networks for Biomedical Image Segmentation*. MICCAI 2015.
3. Wang, X., Peng, Y., Lu, L., Lu, Z., Bagheri, M., & Summers, R. M. (2017). *ChestX-ray8 : Hospital-scale Chest X-ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases*. CVPR 2017.
4. Cohen, J. P., et al. (2022). *TorchXRayVision : A library of chest X-ray datasets and models*. github.com/mlmed/torchxrayvision
5. Flutter Team (2024). *Flutter Documentation*. flutter.dev
6. Google (2024). *TensorFlow Lite — ML for Mobile and Edge Devices*. tensorflow.org/lite
7. National Institutes of Health (2017). *NIH Chest X-ray Dataset*. nihcc.app.box.com
8. Ministère de la Santé du Maroc (2022). *Rapport sur les maladies respiratoires*. sante.gov.ma